



The Open University

M263 Building Blocks of Software

Unit 15

Efficiency

Study guide

Sections 1 and 2 provide an introduction to the issues discussed in this unit, and Section 6 provides a brief review of their significance. These sections provide an important context for the technical material discussed in this unit, but you should not spend a long time on details within these sections. The remaining three sections (Sections 3, 4 and 5) are of about equal length and importance. If you are not confident about algebraic manipulations, there are places where this may cause you some difficulty; but do not worry about such points to any great extent — it is more important to gain an overview of the issue of efficiency and the approach to it covered in this unit.

There are some practical computing activities associated with this unit. You should not spend more than about 10% of your study time for the unit on practical work.

This publication forms part of an Open University course. Details of this and other Open University courses can be obtained from the Course Information and Advice Centre, PO Box 724, The Open University, Milton Keynes, MK7 6ZS, United Kingdom: tel. +44 (0)1908 653231, e-mail general-enquiries@open.ac.uk

Alternatively, you may visit the Open University website at <http://www.open.ac.uk> where you can learn more about the wide range of courses and packs offered at all levels by the Open University.

To purchase a selection of Open University course materials, please visit the Open University webshop at www.ouw.co.uk, or contact Open University Worldwide, Michael Young Building, Walton Hall, Milton Keynes MK7 6AA, United Kingdom for a brochure. tel. +44 (0)1908 858785; fax +44 (0)1908 858787; e-mail ouwenq@open.ac.uk

The Open University, Walton Hall, Milton Keynes, MK7 6AA.

First published 2004. Second edition 2008.

Copyright © 2008 The Open University

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, transmitted or utilised in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without written permission from the publisher or a licence from the Copyright Licensing Agency Ltd. Details of such licences (for reprographic reproduction) may be obtained from the Copyright Licensing Agency Ltd of 90 Tottenham Court Road, London W1T 4LP.

Edited, designed and typeset by The Open University, using the Open University T_EX System.

Printed and bound in the United Kingdom by The Charlesworth Group, Wakefield.

ISBN 978 07492 6851 0

Contents

Introduction	4
1 Taking time	5
2 Algorithmic efficiency	8
3 Counting statement executions	11
3.1 The time complexity function	12
3.2 Sequential search and binary search	20
4 Recurrence systems	28
4.1 Recurrence systems for time complexity	28
4.2 Solving recurrence systems	31
5 Comparison of time complexity functions	35
5.1 Comparing graphs	35
5.2 Big Oh notation	40
6 Postscripts	45
Summary of Unit 15	47
Solutions to the exercises	48
Index	51

Introduction

This unit is about the efficiency of computer programs. The term “efficiency” has many interpretations. For example, it can relate to how quickly a program executes, to how much disk space or memory the program occupies, or to how easy the program is to maintain. The precise meaning of “efficiency”, as used by computer scientists, will become clearer as we progress through this unit. In this unit, we have chosen to focus on the amount of time required for some piece of software to run. This is an aspect of efficiency upon which a software engineer can have a direct impact.

Our view of running time will be a quantitative one: qualitative terms, such as “quite fast” or “pretty slow” would not provide us with enough information.

The amount of time it takes to execute a computer program is influenced by a number of factors, many of which do not depend upon the code itself. Accordingly, we will need to develop a measure of time that depends only upon the code used. Our chosen measure is based upon the number of statements executed when a program runs. This is different to the number of lines in a program because it includes repetitions of statements, for example, those inside **for** or **while** loops. Using this measure of time means that we can compare programs independently of the computer they run on; i.e. our comparisons are **platform independent**.

A platform independent description of efficiency is important because it:

- helps us choose between alternative implementations for a problem;
- aids communication between computer scientists using different hardware;
- tells us something about how hard a problem is to solve;
- may tell us whether we can expect to solve a problem at all.

In Section 1 we look at a range of factors affecting program speed, and provide an informal example of an algorithm whose efficiency (or lack of it!) dominates all other performance factors by a considerable margin.

Section 2 introduces the concept of “time efficiency” as a measure of how much work needs to be done to solve a problem using a particular algorithm. We see that the time required to solve a problem generally depends on the size of the problem, where size is measured in terms of some variable input data relating to the problem.

Section 3 develops a quantitative measure of time efficiency, the time complexity function. This is based on counting statements in code, and can be used to compare different implementations. We look at examples of how to infer the time complexity function of an implementation by analysing its behaviour in relation to the size of the problem.

Section 4 shows how we can develop a more rigorous approach to characterising the time efficiency of an implementation. We shall see how the time complexity function of a recursive implementation can be described using a recurrence system. If we can solve the recurrence system, then we can obtain a formula for the time complexity function in terms of the size of the input.

In Section 5, we provide some comparisons between functions given by formulae that often arise as time complexity functions — and, in particular,

consider how such functions behave as the size of the problem gets large. This allows us to develop a feeling for why some algorithms might be preferred to others. In Section 6, we introduce a mathematical definition used to compare the behaviour of functions $T(n)$ (where n is an integer), as n becomes very large. We can use this to compare the efficiency of alternative implementations when we have formulae for their time complexity functions.

There is rather a lot of mathematical notation used in this unit, and you will need to do some mathematics yourself. If you feel anxious about this, do not worry too much. It is important to read and work through the unit focusing on the wider messages, even if you do not feel able to get to grips with every piece of algebra.

1 Taking time

It is well known that the kind of computer you have affects the time it takes to run software. Here are some of the factors that may affect how long a program takes to run.

1. The amount and kind of memory the computer has. In general, the more, the better. Some kinds of memory are faster to use than others.
2. The processor speed. For example, a 2 GHz processor may be faster than a 1 GHz processor (though there are other factors to consider).
3. The speed with which the system can access the hard drive, if one is being used.
4. Whether the computer is part of a network of computers that share some resource.
5. Whether the computer is running more than one program at the same time. If it is, it may be dividing its time between them.
6. The problem or task that the program is undertaking. We shall see that some problems are, by their very nature, difficult to solve. (Some cannot be solved on *any* digital computer — either now, or in the future!)
7. The size of the input in the problem being addressed. In general, we can identify a variable that tells us the size of the input. For example, if we are finding the sum of a list of numbers, the length of the list is significant.
8. The nature of the data the program is working on.
9. The specific algorithm that the program employs to solve a problem.

In the list above, (1)–(5) relate to the hardware you are using, (6) relates to the problem you are working on, (7) and (8) concern the data upon which your program is operating and (9) relates to the precise method you use to solve a problem. Since we have no influence on the hardware used, in this unit only points (6)–(9) will concern us. The activities in this section will give you a sense for some of the issues we shall be exploring.

We now turn to a specific problem. Let us imagine that a supermarket offers a home delivery service. A driver loads a van with a number of orders to be delivered. A natural question to ask is: what is the shortest route the driver can take, starting and ending at the warehouse, such that each customer is visited on just the one occasion necessary to deliver their order? We can begin to answer this question if we know the distances (measured in kilometres, for example) between all the customers.

GHz stands for gigahertz. The prefix “giga” represents a 10^9 multiplying factor in most cases. However, for computer memory, a gigabyte is technically 2^{30} .

The notion of “shortest” might take into account just the distance, or other factors, such as the travelling time or the cost of fuel.

A program to find the shortest route would be useful to the supermarket. How might we begin to develop an algorithm to do this?

Consider an example, as illustrated in Figure 1.1, in which we have a warehouse, W , and just three customers, represented as 1, 2 and 3. One possible route visits the customers in numerical order and returns to the warehouse. We might write this route as $[W, 1, 2, 3, W]$. We could calculate the distance for this route by summing the distances between each of the locations in order. An alternative route is $[W, 3, 1, 2, W]$, and its distance could be calculated in the same way. An obvious strategy to solve this problem is to find all possible routes, calculate the distance involved in each, and output the shortest route. What are the possible routes, starting and ending at the warehouse (W), and visiting each customer (1, 2 and 3) once?

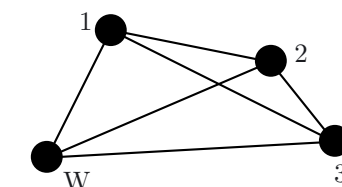


Figure 1.1

Activity 1.1

Use the notation suggested above to write down all possible routes visiting the three customers. How many alternative routes are there?

Discussion The possible routes are given below.

$[W, 1, 2, 3, W]$, $[W, 1, 3, 2, W]$, $[W, 2, 3, 1, W]$,
 $[W, 2, 1, 3, W]$, $[W, 3, 1, 2, W]$, $[W, 3, 2, 1, W]$.

So there are six possible routes.

Activity 1.2

The number of possible routes depends on the number of customers to be visited. For one customer there is just one route: $[W, 1, W]$. In Activity 1.1, we found that there are six routes if there are three customers. Calculate the number of possible routes if the number of customers is: (a) 2; (b) 4.

Do not spend more than two minutes (literally) on this.

Discussion For two customers, there are just two routes $[W, 1, 2, W]$ and $[W, 2, 1, W]$. If you started to list the possible routes when there are four customers, you almost certainly had to stop before the two minutes elapsed! In fact, there are 24 possible routes.

You may have calculated the number of possible routes without having to list them all. One way to do this is as follows. If there are four customers then, starting at W , the driver has four possible destinations for the first visit. Having made the first visit, there are three remaining customers to visit. We saw in Activity 1.1 that there are six routes visiting three customers. We can combine each of the four ways of starting the route with six ways of visiting the three remaining customers, giving a total of $4 \times 6 = 24$ routes.

For $n > 0$, let $ROUTES(n)$ be the number of possible routes visiting n customers. We have $ROUTES(1) = 1$, and, for $n > 1$, there are n customers who might be visited first, and then $ROUTES(n - 1)$ ways of visiting the remaining $n - 1$ customers in each case. So $ROUTES(n)$ must satisfy the following recurrence system.

$$\begin{aligned} ROUTES(1) &= 1 \\ ROUTES(n) &= n \times ROUTES(n - 1) \quad (n > 1) \end{aligned}$$

Now in Unit 13 (page 7), we gave a recurrence system for the function *FACTORIAL*, giving the product of the integers between 1 and n :

$$FACTORIAL(n) = 1 \times 2 \times 3 \times 4 \times \dots \times (n-1) \times n.$$

The recurrence systems for *ROUTES* and for *FACTORIAL* are identical, showing that these functions are the same. So we have $ROUTES(n) = n!$.

$FACTORIAL(n)$ is often written as $n!$, and we shall use this notation here.

Activity 1.3

- (a) Give the number of alternative routes to be considered if there are five customers.

Discussion

- (a) There are $ROUTES(5) = 5!$ possible routes, giving $5 \times 4! = 5 \times 24 = 120$ alternative routes to consider.

Let us now explore what happens as the number of customers increases. If there are 10 customers, then there will be $10!$ possible routes, where

$$10! = 10 \times 9 \times 8 \times 7 \times 6 \times 5 \times 4 \times 3 \times 2 \times 1 = 3\,628\,800.$$

By now you may see a problem that is emerging. Values of $n!$ for various values of n (up to $n = 50$) are shown in Table 1.1.

Number of customers	Number of routes to consider
5	120
10	3 628 800
15	1.31E+12
20	2.43E+18
25	1.55E+25
30	2.65E+32
35	1.03E+40
40	8.16E+47
45	1.20E+56
50	3.04E+64

These values of $n!$ can be found using the WorkPad — use the iterative rather than the recursive implementation of *FACTORIAL*.

The values shown for $n \geq 15$ are rounded to 3 figures.

Note that 1.31E+12 is another way of writing 1.31×10^{12} .

Table 1.1 Values of $n!$ for values of $n = 5, 10, 15, \dots, 50$.

For just fifteen customers, there are about 1.31×10^{12} different routes — that is, over a million million routes. By this time, you may be wondering whether it is really feasible to consider all of these routes in deciding which is shortest.

Activity 1.4

- (a) If there are 50 customers, the number of routes to consider is approximately 3×10^{64} . If a computer can calculate the distance of a single route in one second, and there are roughly 3×10^7 seconds in a year, estimate the time, in years, that it would take to find the shortest route using our strategy.
- (b) If a computer were a million times faster than the one in (a), would this make a meaningful difference? What about a million million times faster?

Discussion

- (a) We can get an estimate of the time, in years, by dividing the number of routes to be considered by the number of seconds in a year:
 $(3 \times 10^{64}) \div (3 \times 10^7) = 10^{57}$ years. We would be waiting quite a while!
 (A millennium is a mere 10^3 years.)
- (b) For practical purposes, these improvements in the speed of a computer make no significant impression on the time taken. A computer that is a million million (i.e. 10^{12}) times faster would still take 10^{45} years to find the shortest route.
-

You may be wondering if there is a better algorithm than the one we have used. We shall return to this question later. In the meantime, you may like to know that the problem we have been looking at is an example of a famous problem known as the Travelling Salesman Problem (or TSP). If you do an internet search you should find plenty of references to it.

Objectives for Section 1

After studying this section you should be able to do the following.

- ☐ Give factors that influence the length of time it takes to run a program.
- ☐ Distinguish between factors that depend on: the computer being used; the problem being addressed; and the algorithm used to solve it.

Exercises on Section 1

Exercise 1.1

A small computer-controlled robot arm is used to drill holes at predetermined locations in electronic circuit boards. It drills one hole at a time. The circuit boards are rectangular, with dimensions 125 mm by 250 mm. If there are 40 holes to be drilled in each board at specified locations, what issues may arise for anyone asked to produce an algorithm to optimise the robot's performance by minimising the amount of time taken to drill all the holes? (Consider how this problem relates to the supermarket delivery problem.)

2 Algorithmic efficiency

We noted in Section 1 that the time taken to execute a program will depend on a number of factors. Some of these depend on the hardware being used, and we shall not consider such factors here. Setting hardware considerations aside, there are still a variety of factors influencing execution times — the programming language used being an example. The actual execution time is not a very portable measure of how good a piece of software is. We want here to develop a view of “time efficiency” that is, so far as possible, independent of considerations relating to the hardware or programming language used.

In Unit 6 we mentioned that we can distinguish between the broad strategy for solving some problem, and the implementation of such a strategy in code. We refer to a broad strategy as an **algorithm**. For example, we looked in Unit 6 at *bubble sort* as a strategy (i.e. algorithm) for the task of sorting a sequence, while in Unit 8 we mentioned briefly the alternative algorithm *heap sort* for the same task. The most portable measure of efficiency is one attached to an algorithm — enabling us to discuss the relative time efficiency of alternative algorithms for the same task, such as bubble sort in comparison to heap sort. However, rather than working with algorithms, we shall consider here implementations of algorithms in course code. These are more concrete, and so easier to analyse. Such an implementation provides an expression of the algorithm and so, in determining the efficiency of the implementation, we will also be characterising the efficiency of the algorithm that it implements. Looking ahead, we shall first (in Section 3) discuss a way of associating a specific measure of time efficiency with an implementation. This measure will depend to some extent on implementation details. Later (in Section 5), we shall introduce a broader measure of time efficiency, and this will characterise the efficiency of the algorithm in a way that should not depend on details of its implementation in course code.

See Unit 6, p. 25.

See Unit 8, p. 43.

Analysis of time efficiency is of particular relevance when discussing problems whose input may be large. In the supermarket delivery problem, we found that when the number of homes became “large”, our algorithm for calculating the number of possible routes was not feasible.

The interpretation of “large” here is likely to depend on the problem under discussion.

As illustrated by the delivery problem, the size of the input to an algorithm is one of the key factors influencing the time it takes for an algorithm to execute. Consequently, we shall aim to describe how much calculation is involved in executing an implementation in terms of the size of the input. (For an algorithm, one might refer to the “problem size”.)

If we were taking a formal approach to measuring efficiency, we would specify an “abstract machine” to execute our algorithms, and the size of the input would be the amount of storage needed to represent the input within this imaginary machine. For example, the number of bits needed to store the number 7 is less than the number of bits needed to store the number 259.

The **Turing Machine** is one such abstract machine. Developed by Alan Turing in 1936, its invention contributed significantly to the creation of the first electronic digital computers.

In fact, we do not need such formality at this level of study and so we can adopt a more informal notion of problem size. Intuitively, the problem size is some factor that influences how much work has to be done. We will want:

1. a way of clarifying “the problem to be solved”;
2. a way of measuring “how much work has been done” in solving it;
3. a way of measuring “the size of the input”; and
4. a way of relating the second point to the third.

To obtain a concrete measure of time efficiency, we shall consider problems in which we evaluate a function given by some specification. In the kind of software we are considering, the *number of statements executed* is often a reliable guide to how long a program will take to compute the required output. Accordingly, we shall identify “how much work has been done” by counting the number of statements that have been executed, and “the size of the input” as a measure associated with the feature of the input that principally determines how many statements are executed.

For example, in the informal algorithm we described for finding the shortest route for the supermarket driver, the number of customers to be visited measures the problem size. As another example, consider searching for an item in a list (for example, a telephone directory). Here, the longer the list,

the more entries that we may have to check. In this case, the length of the list is a good measure of the size of the problem.

Time efficiency is a description geared to what happens as the problem size gets large. If, in practice, only “small” instances of a problem arise, then a simple and less efficient program may be preferred to a complex but more efficient one, because it may be simpler to produce, read and maintain.

Unless you are certain that only small inputs will be used, it would be ill-advised to choose an implementation because it worked well on an instance of a problem that was small, without considering how well it might work when the problem is larger. If the performance of the implementation were to deteriorate severely for large inputs, this could cause serious problems.

Our aim here is to discuss how to describe time efficiency in general, and in particular when the problem size is large. There may be properties of the input data other than its size that can be significant in deciding the time required to solve it. For example, if software is required to search for some entry in a list, and it can be guaranteed to have a *sorted* sequence as input, then the search can be much quicker than if the list were unsorted.

See Unit 6, p. 24 and p. 42.

Activity 2.1

Compare the amount of effort needed for you to find a telephone number in a telephone directory when you are given the name to be found, with the amount of effort needed to find a name in the directory when you are given only the telephone number.

Discussion A telephone directory is arranged so that the names appear in alphabetical order, and we use this to find the required number quickly. Few of us would have the patience to look for a number without a name.

To be precise, the names are in lexicographic order.

In specifying a function to be evaluated, we can add a precondition to require that a sequence input is sorted — such a precondition can influence significantly the time efficiency of possible implementations.

There are three cases to distinguish when running an algorithm: **best case**, **average case** and **worst case**. The best case is the best performance we can expect an algorithm to have, and the worst is the opposite.

Best cases are mainly of interest if we can ensure that our input always, or at least often, meets the requirements for that case to occur. The worst case shows just how badly software may perform. It is *essential* to consider the worst case for software that is “safety critical” (for example, where someone’s life or health is at stake) or “mission critical” (for example, where jobs and livelihoods may be in jeopardy).

In many cases, we would like to know how our software will behave “on average”. If a process is performed very frequently, then we may accept the occasional relatively slow execution to achieve a quicker “typical” execution time. However, “average case” is harder to analyse. It depends on the relative frequency with which different versions of the problem need to be solved. We can estimate the “average performance” only if we can make satisfactory predictions about the distribution of such cases.

Even if we can, the mathematical analysis of “average case” performance is more difficult.

Best cases may not be different from worst cases. Sometimes the two are the same; in that case the average case is also the same. In what follows, we shall concentrate on worst cases.

Objectives for Section 2

After studying this section you should be able to do the following.

- ☐ Distinguish between time efficiency and actual running time.
- ☐ Describe informally the amount of work done by an algorithm.
- ☐ Distinguish between best, worst and average cases for time efficiency.

Exercises on Section 2

Exercise 2.1

A prime number is an integer p (greater than 1) that is exactly divisible only by 1 and p . A function to test for primality takes as input an integer z that is greater than 1, and returns **true** if z is prime and **false** otherwise.

Suppose that an implementation of this function employs the following algorithm (which is far from the best). Starting at 2, and working up to $z - 1$ if necessary, divide z by each integer in turn. Return **false** and stop the repetition if one of these divisions results in a remainder of zero, and return **true** otherwise.

- (a) Suggest a measure of the problem size here.
- (b) Suggest best and worst cases for this implementation. How many divisions are required in each case?

Exercise 2.2

A computer system administrator keeps information on users who are currently logged on so that the status of any account can be checked. Without attempting any detailed calculations, consider the advantages and disadvantages of the following two approaches to recording this information from the point of view of the efficiency of a program that must search to see if a particular user is logged on.

- (a) The account numbers of all logged on users are stored in an unsorted sequence (a vector, say), which is extended or contracted as the number of users logged on changes, as in the example below.

[arw4455, ttp1009, hjc8532, epc3476]

- (b) The status of each valid user is stored in a table in which all account holders are given in lexicographic order, and with an associated integer to show whether or not each account holder is logged on: 0 means the account holder is logged off and 1 means they are logged on. An example of such a table is given in Table 2.1.

asj9767	0
arw4455	1
epc3476	1
fhh9878	0
gtv3542	0
hjc8532	1
rdf5480	0
ttp1009	1
tjh5297	0
vfd3324	0

Table 2.1

3 Counting statement executions

We will now develop a specific measure of the time cost of an implementation. As noted in Section 2, we shall consider a problem in which the task is to evaluate a function as given in a specification. We shall associate with any implementation of the function a measure of the time cost of running it. For an implementation, say A , we can define the associated measure of

time cost as follows. Let $T_A(n)$ be the total number of statement executions when A is executed with:

- an input of size n ;
- in the worst case.

We will call $T_A(n)$ the **time complexity function** of A . (It would be more precise to say “worst case time complexity function”, but we shall not consider other approaches here.)

Note that the time complexity function is also known as the **time efficiency function**.

Note that $T_A(n)$ should count repetitions: if a particular code statement is repeated 100 times, this contributes 100 to the count given by $T_A(n)$.

The way we measure the size of the input will depend on the form of data being input. Also, different inputs of the same size may involve different numbers of statement executions: for example, when searching a list of length n , we may find the required item very quickly — or we may need to search through the entire list. We take $T_A(n)$ to be the count in the worst case; that is, the largest number of statement executions needed for any permitted input of size n .

A permitted input is a value of the type given in the signature line of the specification, and satisfying the precondition.

This approach involves an element of approximation. We are concerned with how long an algorithm takes to run, and so it would be natural to ask how long each individual statement in a program might take to execute. It is possible to estimate these times, and do a detailed calculation of how long an implementation would take to execute. This detailed analysis could be useful in a system where the hardware is fixed, and for “real-time” applications, but such timed results are not portable to different hardware, or even the same hardware under different conditions.

In real time applications, such as in avionics systems, for example, the response time may be critical to success.

Our approach effectively assumes that all statements take about the same amount of time to execute. Of course, this is not true, but it turns out to be a reasonable approximation. The execution time of an algorithm tends to reflect the total number of code statements that are executed (taking into account repetitions), setting aside exactly what each line of code does.

Our approach is of more general applicability.

The definition given above states precisely what we will mean by the time complexity function for an implementation expressed in course code. We shall not give a formal definition of the idea in the context of algorithms not expressed in code. However, we will use the idea in an informal way. For example we would say that the algorithm given in Section 1 for the supermarket delivery problem has a time complexity function $T(n) = n!$.

3.1 The time complexity function

We shall now look at some examples where we will count how often each statement of an implementation is executed, and consider how the count will depend on the size of the input. We shall then infer expressions for $T(n)$, which describe the relationship between the problem size and the work done by the implementation.

In arriving at $T(n)$, we shall count executions of all the code statements, including **var** and **return** statements. There are some parts of our code that we do not count as statements. In particular, we do not count a line with only a curly bracket as a statement, because these brackets are just part of the syntax of the language; they delineate parts of the implementation, and are not executable statements. Nor do we count the header line, which again is syntax. Likewise we would not count a comment as a statement — comments are not executed.

EXAMPLE 3.1

Below we give the specification of a function *INTSUM* and an iterative implementation *intSum1* of this function.

function *INTSUM*(*n* in Int) **return** in Int

pre $n > 0$.

post The returned value is the sum of the integers between 1 and *n*.

```
function INTSUM(n)
{
    //intSum1
    var result in Int           // (1)
    var index in Int           // (2)
    result <-- 0                // (3)
    for (index <-- 1 to n)      // (4)
    {
        result <-- result + index // (5)
    }
    return result              // (6)
}
```

In this implementation there are six executable statements, labelled (1)–(6). Some statements are executed just once; others are executed with a frequency that is governed by the input value, *n*.

To count the total number of statement executions, we will trace the execution of the implementation in a very careful way. In the table below, every statement execution is shown, in the case when the input, *n*, is 3.

In a **for** loop, there are statements forming the loop body (in *intSum1*, just (5)), together with the **for** statement itself (in *intSum1*, (4)). In our total count, each of these adds a statement execution for each execution of the loop body. We shall also add one final execution of (4) when the loop is left.

Statement		<i>result</i>	<i>index</i>
(1)	var <i>result</i> in Int	—	—
(2)	var <i>index</i> in Int	—	—
(3)	<i>result</i> $\leftarrow$ 0	0	—
(4)	<i>index</i> $\leftarrow$ 1	0	1
(5)	<i>result</i> $\leftarrow$ <i>result</i> + <i>index</i>	1	1
(4)	<i>index</i> $\leftarrow$ 2	1	2
(5)	<i>result</i> $\leftarrow$ <i>result</i> + <i>index</i>	3	2
(4)	<i>index</i> $\leftarrow$ 3	3	3
(5)	<i>result</i> $\leftarrow$ <i>result</i> + <i>index</i>	6	3
(4)	loop ends	6	—
(6)	return <i>result</i>	6	—

This reflects the pass when the **for** statement is examined and it is recognised that the loop body should cease execution.

We shall count **while** loops in a similar way.

Reading down the left-most column of the table we can see that statements (1), (2), (3) and (6) are each executed just once. Statement (5) is executed three times and (4) four times. Here, we traced the algorithm when *n* is 3, and found that there are 11 statement executions in total. In general, the number of times that (4) and (5) are executed will depend on *n*. We can infer that for input *n*, (5) will be executed *n* times and (4) will be executed *n* + 1 times. So for a general input, *n*, the total number of statement executions is:

$$1 + 1 + 1 + (n + 1) + n + 1 = 2n + 5.$$

Note that $2 \times 3 + 5 = 11$.

Each term on the left-hand side of the above equation shows the number of times one of the numbered statements in the implementation is executed: 1 for (1); 1 for (2), 1 for (3), $n + 1$ for (4); n for (5), and 1 for (6). (We shall follow a similar approach below when writing expressions for the total number of statement executions.)

So the time complexity function for the implementation *intSum1* is:

$$T_{\text{intSum1}}(n) = 2n + 5.$$

We give below an alternative implementation of *INTSUM*, *intSum2*. This uses a result from Unit 14, that $1 + 2 + 3 + \dots + n = n(n + 1)/2$.

```
function INTSUM(n)
{
    //intSum2
    var sum in Int          // (1)
    sum <-- DIV(n*(n+1),2)  // (2)
    return sum              // (3)
}
```

This implementation has just three statements. Each of them is executed exactly once, regardless of the input. We can see that: $T_{\text{intSum2}}(n) = 3$.

Comparing the time complexity functions for the two implementations *intSum1* and *intSum2*, we see that the time taken to execute *intSum1* will grow as n grows, but that the time taken to execute *intSum2* is unaffected by n . Clearly, *intSum2* is the preferred algorithm.

Note that each time complexity function in Example 3.1 is associated with an implementation, not with the “problem” being solved. Each of *impSum1* and *impSum2* implements the same function *INTSUM* — but they have quite different time complexity.

Activity 3.1

A function *SQUARE* is specified below, with an implementation *square1*.

function *SQUARE*(n in Int) **return** in Int

pre $n \geq 0$.

post The returned value is the square of n (i.e. $n^2 = n \times n$).

```
function SQUARE(n)
{
    //square1
    var result in Int          // (1)
    var index in Int           // (2)
    result <-- 0                // (3)
    for (index <-- 1 to n)      // (4)
    {
        result <-- result + n  // (5)
    }
    return result              // (6)
}
```

There is no issue of “worst” case to consider here. With the input an integer n , there is only one input “of size n ” to consider.

See Unit 14, Result 2.3.

Do not worry about why we might want such an implementation — treat this as an exercise in counting statement executions.

A trace table when the input n is 3 is started below. Complete the trace table and determine the number of statement executions for that input. Then suggest the number of statement executions for a general input, n .

Statement		<i>result</i>	<i>index</i>
(1)	var <i>result</i> in Int	—	
(2)	var <i>index</i> in Int	—	—
(3)	<i>result</i> $\leftarrow$ 0	0	—
(4)	<i>index</i> $\leftarrow$ 1	0	1
(5)	<i>result</i> $\leftarrow$ <i>result</i> + <i>n</i>	3	1

Discussion The trace table when the input *n* is 3 is completed below.

Statement		<i>result</i>	<i>index</i>
(4)	<i>index</i> $\leftarrow$ 2	3	2
(5)	<i>result</i> $\leftarrow$ <i>result</i> + <i>n</i>	6	2
(4)	<i>index</i> $\leftarrow$ 3	6	3
(5)	<i>result</i> $\leftarrow$ <i>result</i> + <i>n</i>	9	3
(4)	loop ends	9	—
(6)	return <i>result</i>	9	—

Statements (1), (2), (3) and (6) are each executed just once; (5) is executed three times, and (4) is executed four times. For a general input *n*, (5) will be executed *n* times and (4) will be executed *n* + 1 times. We can infer that, for input *n*, the total number of statement executions is: $1 + 1 + 1 + (n + 1) + n + 1 = 2n + 5$.

So we may write our time complexity function for this particular algorithm as: $T_{\text{square1}}(n) = 2n + 5$.

This is similar to the algorithm *intSum1*, above. (Indeed, from a time complexity perspective, it is identical.)

Activity 3.2

An alternative implementation, *square2*, of the function *SQUARE* is given below, followed by the first part of a trace table for *square2* when the input *n* is 3. Complete the trace table, and so determine the number of statements executed for this input.

How many statements will be executed for a general input *n*?

Which of *square1* and *square2* do you think is “more efficient”?

Again, do not wonder why we might want such an implementation — treat it as an exercise in counting statement executions.

We could simply use a single statement **return** *n* * *n* to implement *SQUARE*.

```
function SQUARE(n)
{
    // square2
    var result in Int           // (1)
    var indexa in Int          // (2)
    var indexb in Int          // (3)
    result <-- 0                // (4)
    for (indexa <-- 1 to n)     // (5)
    {
        for (indexb <-- 1 to n) // (6)
        {
            result <-- result + 1 // (7)
        }
    }
    return result              // (8)
}
```

A trace table for the case $n = 3$ starts as follows:

Statement		<i>result</i>	<i>indexa</i>	<i>indexb</i>
(1)	var <i>result</i> in Int	—		
(2)	var <i>indexa</i> in Int	—	—	
(3)	var <i>indexb</i> in Int	—	—	—
(4)	<i>result</i> $\leftarrow 0$	0	—	—
(5)	<i>indexa</i> $\leftarrow 1$	0	1	—
(6)	<i>indexb</i> $\leftarrow 1$	0	1	1
(7)	<i>result</i> $\leftarrow$ <i>result</i> + 1	1	1	1
(6)	<i>indexb</i> $\leftarrow 2$	1	1	2
(7)	<i>result</i> $\leftarrow$ <i>result</i> + 1	2	1	2
(6)	<i>indexb</i> $\leftarrow 3$	2	1	3
(7)	<i>result</i> $\leftarrow$ <i>result</i> + 1	3	1	3
(6)	inner loop ends	3	1	—
(5)	<i>indexa</i> $\leftarrow 2$	3	2	—
(6)	<i>indexb</i> $\leftarrow 1$	3	2	1
(7)	<i>result</i> $\leftarrow$ <i>result</i> + 1	4	2	1
(6)	<i>indexb</i> $\leftarrow 2$	4	2	2
(7)	<i>result</i> $\leftarrow$ <i>result</i> + 1	5	2	2
(6)	<i>indexb</i> $\leftarrow 3$	5	2	3
(7)	<i>result</i> $\leftarrow$ <i>result</i> + 1	6	2	3
(6)	inner loop ends	6	2	—

Discussion The trace table when n is 3 is completed below.

(5)	<i>indexa</i> $\leftarrow 3$	6	3	—
(6)	<i>indexb</i> $\leftarrow 1$	6	3	1
(7)	<i>result</i> $\leftarrow$ <i>result</i> + 1	7	3	1
(6)	<i>indexb</i> $\leftarrow 2$	7	3	2
(7)	<i>result</i> $\leftarrow$ <i>result</i> + 1	8	3	2
(6)	<i>indexb</i> $\leftarrow 3$	8	3	3
(7)	<i>result</i> $\leftarrow$ <i>result</i> + 1	9	3	3
(6)	inner loop ends	9	3	—
(5)	outer loop ends	9	—	—
(8)	return <i>result</i>	9	—	—

Statements (1), (2), (3), (4) and (8) are each executed just once. For a general input n , statement (5) is executed $n + 1$ times. The body of the outer loop is executed n times and, within each of these, (7) is executed n times and (6) is executed $n + 1$ times. So (7) is executed a total of $n \times n = n^2$ times, and (6) is executed $n \times (n + 1) = n^2 + n$ times.

The total number of statement executions for input n will be:

$$1 + 1 + 1 + 1 + (n + 1) + (n^2 + n) + n^2 + 1.$$

Thus

$$T_{\text{square2}}(n) = 2n^2 + 2n + 6.$$

As a check, this formula gives $T(3) = 2 \times 3^2 + 2 \times 3 + 6 = 30$, while the trace above with n equal to 3 contains 30 statement executions.

Of these two implementations, *square1* certainly seems the more efficient!

We shall now compare the time complexity functions of *square1* and *square2*.

Activity 3.3

Calculate the values of the two time complexity functions: $2n + 5$ (for *square1*) and $2n^2 + 2n + 6$ (for *square2*), when n takes each of the values 1, 2, 3, 4, 5 and 10. Tabulate the values in each case, and compare the way they change as n gets larger.

Discussion The values are tabulated below.

n (input size)	$2n + 5$ (for <i>square1</i>)	$2n^2 + 2n + 6$ (for <i>square2</i>)
1	7	10
2	9	18
3	11	30
4	13	46
5	15	66
10	25	226

We can see that the total number of statements executions for *square2* is growing much faster than is the case for *square1*. As you did the calculations, you may have noticed that it is the term in n^2 that causes the value of T_{square2} to increase so much faster than T_{square1} .

Remember our simplifying assumption that all statements take the same time to execute — without it, the comparison above is not meaningful.

A function that returns the total of a sequence of integers stored as a vector is specified below. We shall compare two implementations of this function.

These implementations were given in Unit 13, p. 23.

function *ADDALL*(v in Vector of Int) return in Int

pre true.

post The returned value is the sum of all the integers in v . (If v is empty, the returned value is 0.)

Activity 3.4

An implementation *itAddAll* of *ADDALL* is given below. Suggest an expression for T_{itAddAll} .

```
function ADDALL(v)
{
    //itAddAll
    var i in Int                // (1)
    var sum in Int              // (2)
    sum <-- 0                    // (3)
    for (i <-- 1 to v.size())    // (4)
    {
        sum <-- sum + v.at(i)    // (5)
    }
    return sum                  // (6)
}
```

Discussion Here, we must first observe that the input is a sequence. We shall measure the size of the input, written n as usual, as the length of the sequence (given by the method *size* of the class **Vector**). For a sequence input, we need to consider whether there may be any difference in behaviour for different inputs of the same size. For this implementation, there is none — all input sequences of size n require the same number of statement executions.

The analysis is similar to that for T_{square1} . Each of statements (1), (2), (3) and (6) will be executed just once. The loop body will execute once for each value of i from 1 to n (the size of the input vector v). So (5) will be executed n times and (4) will be executed $n + 1$ times. We obtain:

$$T_{\text{itAddAll}}(n) = 1 + 1 + 1 + (n + 1) + n + 1 = 2n + 5.$$

You may notice that *ADDALL* has precondition **true**, so an empty sequence is a valid input. We can include the case where n is zero in the time complexity function. The formula $T(n) = 2n + 5$ gives $T(0) = 5$, and there are five statement executions in that case — the body of the **for** loop is never executed.

We now consider the time efficiency of a recursive implementation of *ADDALL*. How can we determine the number of statements executed for a recursive implementation? We saw in Unit 13 how we can follow such an implementation for a particular input value. However, we need to trace the evaluation in a more detailed way here. We need to be sure to keep track of *every* statement execution, so we can keep a count.

EXAMPLE 3.2

A recursive implementation *recAddAll* of the function *ADDALL* is given below. We shall follow the execution of this when the input v is $[3, 5, 10]$. We want to locate every statement execution, for our count.

This implementation has an **else** in it. We do not count the **else** as a statement, because it is just syntax to identify a branch in the program, and is not executable.

```
function ADDALL(v)
{
    //recAddAll
    var temp in Vector of Int    // (1)
    var res in Int              // (2)
    temp <-- v.clone()          // (3)
    if (temp.size() == 0) then  // (4)
    {
        res <-- 0               // (5)
    }
    else
    {
        res <-- temp.last()     // (6)
        temp.removeLast()      // (7)
        res <-- res + ADDALL(temp) // (8)
    }
    return res                  // (9)
}
```

A trace showing every statement execution when the input vector v is $[3, 5, 10]$ is started below.

When v is $[3, 5, 10]$:

Statement	Comment	$temp$	res
(1)		$[]$	
(2)		$[]$	—
(3)		$[3, 5, 10]$	—
(4)	$3 == 0$ is false	$[3, 5, 10]$	—
(6)		$[3, 5, 10]$	10
(7)		$[3, 5]$	10
(8)	pending evaluation	$[3, 5]$	$10 + ADDALL([3, 5])$

The pending evaluation of $ADDALL([3, 5])$ will be found using $recAddAll$. So we now need to trace evaluation of $ADDALL(v)$ when v is $[3, 5]$.

When v is $[3, 5]$:

Statement	Comment	$temp$	res
(1)		$[]$	
(2)		$[]$	—
(3)		$[3, 5]$	—
(4)	$2 == 0$ is false	$[3, 5]$	—
(6)		$[3, 5]$	5
(7)		$[3]$	5
(8)	pending evaluation	$[3]$	$5 + ADDALL([3])$

There is still a pending evaluation — $ADDALL([3])$.

Activity 3.5

Complete the evaluation started in Example 3.2, and determine the total number of statement executions required when $recAddAll$ is used to find $ADDALL(v)$ when v is $[3, 5, 10]$.

Then try to suggest an expression for the number of statements executed by $recAddAll$ when the input vector has size n .

Discussion When v is $[3]$:

Statement	Comment	$temp$	res
(1)		$[]$	
(2)		$[]$	—
(3)		$[3]$	—
(4)	$1 == 0$ is false	$[3]$	—
(6)		$[3]$	3
(7)		$[]$	3
(8)	pending evaluation	$[]$	$3 + ADDALL([])$

And there is still a pending evaluation — $ADDALL([])$.

When v is $[]$:

Statement	Comment	$temp$	res
(1)		$[]$	
(2)		$[]$	—
(3)		$[]$	—
(4)	$0 == 0$ is true	$[]$	—
(5)		$[]$	0
(9)	return 0	$[]$	0

We are now able to complete the pending evaluations, each of which will involve an execution of (9).

With input [3]: return $3 + 0 = 3$ (statement 9).

With input [3, 5]: return $5 + 3 = 8$ (statement 9).

With input [3, 5, 10]: return $10 + 8 = 18$ (statement 9).

In all, this trace involves $3 \times 8 + 6 = 30$ statement executions.

How can we determine the number of executions for a general input vector of size n ? The example above suggest a general pattern. There are n calls involving a pending recursive call. Each of these involves execution of the eight statements (1)–(4) and (6)–(9). Then there is a final call involving the stopping condition. This requires six statement executions: (1)–(5) and (9). So we will have a total of $8n + 6$ statement executions, i.e.

$$T_{\text{recAddAll}}(n) = 8n + 6.$$

As with *itAddAll*, the number of statement executions will be the same for any input vector of size n .

You may like to check what this formula gives when n is 3.

Here is a table to give us some idea of the relative amounts of time to execute *itAddAll* and *recAddAll* for different values of n , based on the time complexity functions determined above.

n (input size)	$2n + 5$ (for <i>itAddAll</i>)	$8n + 6$ (for <i>recAddAll</i>)
1	7	14
2	9	22
3	11	30
4	13	38
5	15	46
10	25	86

The implementation *itAddAll* is clearly more efficient according to our measure. However, this comparison depends heavily on our assumption that each statement takes the same time to execute.

Another issue to bear in mind is that recursive implementations carry an overhead. Recursive calls generally require the storage of pending information, typically on a stack. Not only does this carry a small time penalty but, in certain cases, the storage required can accumulate to the point where the program cannot continue running! For this reason one would probably prefer *itAddAll* anyway.

3.2 Sequential search and binary search

We now consider the task of finding whether a word appears in a dictionary, where the dictionary is stored as a sequence of strings. We will give implementations based on two very different approaches to the problem, which were mentioned in Unit 6. The first is to examine the items one at a time, starting at the front and working through to the end. The second depends on the dictionary holding the words in lexicographic order, and is based on “halving”.

See Unit 6, p. 42.

We look first at a search examining the items in turn, starting at the front (**sequential search** or **linear search**). A corresponding function is specified below.

function *ISINSEQ*(*s* in **mString**, *dictionary* in **Vector of mString**) **return** in **Bool**

pre *dictionary* is not empty.

post The returned value is **true** if *s* appears in *dictionary*, and is **false** otherwise.

An implementation *seqSearch*, using sequential search, is given below.

```
function ISINSEQ(s,dictionary)
{
    //seqSearch
    var found in Bool                // (1)
    var index in Int                 // (2)
    var done in Bool                 // (3)
    found <-- false                  // (4)
    done <-- false                   // (5)
    index <-- 1                      // (6)
    while (NOT(found) /\ NOT(done))  // (7)
    {
        if (s.equal(dictionary.at(index))) then // (8)
        {
            found <-- true           // (9)
        }
        else
        {
            if (index == dictionary.size()) then // (10)
            {
                done <-- true        // (11)
            }
            else
            {
                index <-- index + 1  // (12)
            }
        }
    }
    return found                    // (13)
}
```

This implementation has branches in it (indicated by the word **else**). Because there are conditions tested in this code, the statements executed depend on the input. This is a hint that this implementation may perform relatively well or poorly for different inputs; in other words, that there may be a “worst case” to consider.

Suppose, for example, that *dictionary* is the dictionary

{“aardvark”, “aardvogel”, “zebra”, “tiger”, “lion”, “gazelle”}.

In *seqSearch*, the items in *dictionary* are examined one at a time, starting at the front, and we stop if the item being looked for is found. Which items will be examined if *s* is “zebra”, or if *s* is “tortoise”?

In the first case, we will only examine the first three items in *dictionary* (i.e. “aardvark”, then “aardvogel”, then “zebra”), and then **true** will be returned.

In the second case, all the items in *dictionary* will be examined. Since none is equal to “tortoise”, **false** will be returned.

The function *ISINSEQ* does not require the input dictionary to be in lexicographic order.

Activity 3.6

- (a) What is an appropriate measure of the problem size for *ISINSEQ*?
- (b) Given a dictionary and a word to search for, what are the best and worst cases (in terms of the number of steps required) for *seqSearch*?
- (c) Find an expression for $T_{seqSearch}(n)$. (Remember that this refers to the worst case.)

Discussion

- (a) The number of words in the dictionary tells us how many words we might have to look at to determine if the word we are seeking is present. So we will take n to be the size of the dictionary, i.e. *dictionary.size()*.
- (b) The best case is when the word we want is the first we look at. The worst case occurs if the word we are looking for is not in the dictionary.
- (c) In every case, statements (1)–(6) and (13) are each executed once. In the worst case, the loop body will be executed n times — once for each item in *dictionary*, and so (8) and (10) are each executed n times. In each execution of the loop body except for the last, (12) will be executed. In the last execution, (11) is executed and (12) is not. So (11) is executed once, and (12) is executed $n - 1$ times. Finally, (7) is executed $n + 1$ times, including one final execution when the **while** condition is evaluated and found to be **false**.

In the worst case, (9) is never executed.

So, in total, the number of statement executions in the worst case is:

$$1 + 1 + 1 + 1 + 1 + 1 + (n + 1) + n + 0 + n + 1 + (n - 1) + 1 = 4n + 8.$$

So $T_{seqSearch}(n) = 4n + 8$.

The alternative approach to searching is called **binary search**. This proceeds in a way similar to that which many people would use to look for a word in a dictionary (or a name in a telephone directory, and so on.). When you open the dictionary, you see if the entry you are looking for is on the current page. If it is not, you decide whether it is alphabetically before or after the words on the page you are looking at. If it is before, you open the dictionary at an earlier page; if it is after, you open the dictionary at a later page. You repeat this until you find the entry, or are convinced it is not there.

Binary search is a little more exact than this. Imagine that every “page” has just one word on it, and that we always choose exactly the middle page of the section of the book in which we are looking for our target word. If there are n pages, the first page we look at is numbered $DIV(n, 2)$. If the word we want comes before the word on the middle page in lexicographic order, then we look at the first part of the dictionary; if it comes after then we look in the second part.

By using *DIV*, we ensure that we get an integer as the “page number”.

The two parts arising from our division may differ in size by one page.

Below, we give a recursive implementation (*impBinSearch*) of the binary search strategy. The function *BINSEARCH* that we shall implement here has a couple of additional inputs, which assist in expressing the recursive approach. These (*low* and *high* in the specification) indicate the “page” numbers of the section of the *dictionary* (itself stored in a vector) in which we are searching. As before, we are searching for an item s in a dictionary *dictionary*. To use *BINSEARCH* to search for s in the whole dictionary, we

can call the function with *low* equal to 1 and *high* equal to the length of the vector *dictionary*.

In this case, it is essential that the dictionary is sorted according to the lexicographic order of strings. This requirement appears as a precondition in the specification of the function *BINSEARCH*.

function *BINSEARCH*(*s* in *mString*, *dictionary* in *Vector* of *mString*, *low* in *Int*, *high* in *Int*) **return** in *Bool*

pre *dictionary* is in lexicographic order.

dictionary is not empty.

$1 \leq low \leq dictionary.size()$.

$1 \leq high \leq dictionary.size()$.

post The returned value is **true** if *s* appears in *dictionary* between the position with index *low* and the position with index *high* (inclusive), and is **false** otherwise. If $low > high$, then **false** is returned.

```
function BINSEARCH(s,dictionary,low,high)
{
    //impBinSearch
    var found in Bool // (1)
    var mid in Int // (2)
    if (low > high) then // (3)
    { /* s cannot appear between the two index positions */
        found <-- false // (4)
    }
    else
    {
        mid <-- DIV((low + high),2) // (5)
        if (s.equal(dictionary.at(mid))) then // (6)
        { /* s has been found */
            found <-- true // (7)
        }
        else
        { /* if the middle word < s */
            if (LESS_THAN(dictionary.at(mid),s)) then // (8)
            { /* look for s in second part of dictionary */
                found <-- BINSEARCH(s,dictionary,mid+1,high) // (9)
            }
            else
            { /* s must be in the first part, if at all */
                found <-- BINSEARCH(s,dictionary,low,mid-1) // (10)
            }
        }
    }
    return found // (11)
}
```

The function *LESS_THAN*(*s*, *t*) compares two variables of type **mString** using the lexicographic order.

LESS_THAN was introduced in Unit 8, p. 33. This function is available in the WorkPad.

Activity 3.7

When $low \leq high$, what form of input provides the “best case” for *impBinSearch*?

How many statements are executed in the best case?

Is the best case affected by the size of the dictionary?

Discussion If $low \leq high$, then the best case is when the word (s) we are looking for is in the middle of the dictionary, and so is the first that we look at in statement (6). In this case, there are no recursive calls, only the stopping condition in (6) applies. Each of statements (1), (2), (3), (5), (6), (7) and (11) is executed once.

The number of statements executed in this best case is 7.

This is independent of the size of the dictionary.

We now turn to the worst case. The task of finding a formula for $T_{impBinSearch}$ raises some awkward technicalities. We shall indicate how we can go about finding such a formula, without going into all the details. The first difficulty lies in what we mean by “the size of the input” in this case. The recursive calls vary low and $high$, and so our measure of the problem size needs to include these. We shall take the problem size n to be given by the number of items being searched, i.e. $high - low + 1$. (A typical call of *BINSEARCH* will have low equal to 1 and $high$ equal to the size of *dictionary*, and in such a case this definition of n coincides with the size of *dictionary*.)

In order to follow the process of repeated halving, it turns out to be much easier if we start with a dictionary whose size is $2^i - 1$ for some integer i , and we shall restrict the analysis to this situation. Suppose, for example, that we start with a *dictionary* of size 15 (i.e. $2^4 - 1$), with $low = 1$ and $high = 15$, and we are searching for an item s that is not in the dictionary, and is larger than any of the items in *dictionary*. In this case, statements (1), (2), (3), (5), (6), (8) and (9) are executed, and (11) will be (after the pending recursive call has been completed). Also, there is a recursive call of *BINSEARCH*($s, dictionary, 9, 15$) to be completed (from (9)). This leaves $15 - 9 + 1 = 7$ items to be searched, and $7 = 2^3 - 1$.

In this case, mid is set to $(1 + 15)/2 = 8$ at (5).

The evaluation of *BINSEARCH*($s, dictionary, 9, 15$) will involve executions of the same eight statements, and leaves a pending recursive call at (9) of *BINSEARCH*($s, dictionary, 13, 15$). So this leaves $15 - 13 + 1 = 3$ items to be searched, and $3 = 2^2 - 1$.

In this case, mid is set to $(9 + 15)/2 = 12$ at (5).

The evaluation of *BINSEARCH*($s, dictionary, 13, 15$) will involve executions of the same eight statements, and leaves a pending recursive call at (9) of *BINSEARCH*($s, dictionary, 15, 15$). So this leaves 1 item to be searched, and $1 = 2^1 - 1$.

In this case, mid is set to $(13 + 15)/2 = 14$ at (5).

The evaluation of *BINSEARCH*($s, dictionary, 15, 15$) will involve executions of the same eight statements, and leaves a pending recursive call at (9) of *BINSEARCH*($s, dictionary, 16, 15$). So this leaves no items to be searched — a stopping case for the recursion.

In this case, mid is set to $(15 + 15)/2 = 15$ at (5).

Finally, evaluation of *BINSEARCH*($s, dictionary, 16, 15$) will involve executions of statements (1), (2), (3), (4) and (11). The value **false** will be returned for *BINSEARCH*($s, dictionary, 16, 15$), and then each pending recursive call will return this same value:

for *BINSEARCH*($s, dictionary, 15, 15$);
then for *BINSEARCH*($s, dictionary, 13, 15$);
then for *BINSEARCH*($s, dictionary, 9, 15$);
and finally for *BINSEARCH*($s, dictionary, 1, 15$).

In this example, eight statements have been executed four times each, and then five further statement in the stopping case, making a total of $4 \times 8 + 5 = 37$ statement executions.

This is an example of the worst case. Let us see how we may generalise this.

Activity 3.8

Suppose that *dictionary* has size $2^i - 1$, where i is a positive integer, and that *low* is equal to 1 and *high* is the size of *dictionary*. Also, suppose that s is larger than any of the items in *dictionary*. Suggest an expression for the number of statements executed by *impBinSearch* in this case.

Discussion In this case there will be i executions of the eight statements: (1), (2), (3), (5), (6), (8), (9) and (11), and then one execution (in the stopping case) of the five statements: (1), (2), (3), (4) and (11).

So in total, there will be $8i + 5$ statement executions.

Now in the case considered in Activity 3.8, the size of the problem is $n = 2^i - 1$. Since the situation considered there is the worst case, we see that (for an input of the appropriate size), we will have

$$T_{\text{impBinSearch}}(2^i - 1) = 8i + 5.$$

To turn this into a formula for $T_{\text{impBinSearch}}(n)$, we first recall logarithms to the base 2. We have $\log_2(n) = x$ if $n = 2^x$. (In words, the log to the base 2 of n is the power to which 2 must be raised to give the number n . For example, $\log_2(8) = 3$ because $8 = 2^3$. Remember also that $2^0 = 1$, so that $\log_2(1) = 0$.)

Now we have, for the problem size, $n = 2^i - 1$. Hence, $n + 1 = 2^i$, and so $i = \log_2(n + 1)$. Consequently:

$$T_{\text{impBinSearch}}(n) = 8 \log_2(n + 1) + 5.$$

We have only derived this expression for the case where the problem size is of the form $n = 2^i - 1$, but this is enough to enable us to compare the efficiency of the two tactics of linear and binary search as the size of the problem increases.

Activity 3.9

Calculate each of $T_{\text{seqSearch}}$ and $T_{\text{impBinSearch}}$ for the following values of n : 1, 3, 7, 15, 31, 63, 127. Comment upon the relative performance of the two algorithms. (Note that these values of n avoid the need for a calculator to find $\log_2(n)$, because $\log_2(1) = 0$, $\log_2(2) = 1$, $\log_2(4) = 2$, etc.)

Discussion The values are tabulated below.

Input size, n	$4n + 8$ (for <i>seqSearch</i>)	$8 \log_2(n + 1) + 5$ (for <i>impBinSearch</i>)
1	12	13
3	20	21
7	36	29
15	68	37
31	132	45
63	260	53
127	516	61

When n is small, the performance of the two implementations is similar — indeed, *seqSearch* executes fewer statements when n is 1. But as n increases in size, there is an increasing difference between the two implementations: with a problem size of 127, *seqSearch* executes 516 statements compared with 61 for *impBinSearch*.

As usual, comparing time complexities in this way should be done with caution, bearing in mind that we may not be comparing like with like, due to the simplifying assumptions we made. However, completing the table above suggests a significant difference between the performance of the two implementations when the problem size is large.

Different implementations of the two algorithms, sequential search and binary search, are possible. The time complexity functions of different implementations of each algorithm will not be identical, but they will share a key general feature: that for sequential search the number of statement executions grows linearly with the size of the problem, while for binary search, the number of statement executions grows like the logarithm of the problem size. This difference will become very significant if the problem size is large. There may be reasons to seek an alternative implementation of the general tactic (for example, one might wish to avoid the use of recursion), but an implementation based on binary search can be expected to be superior in a situation where one is regularly searching in large collections.

Binary search requires that the sequence of words forming the dictionary is sorted, while linear search does not. There will be a time cost involved in sorting the words into dictionary order. However, to do this is a “one-off” task, while searching for a word in a dictionary is a task that will be performed frequently.

In this section we have developed a quantitative approach to time complexity that allows us to compare alternative implementations of the same problem. We have done this by measuring time complexity as the total number of statement executions when the implementation is run. This approach is independent of the computing hardware used.

The time complexity function of an implementation gives the total number of statement executions expressed in terms of the size of the input, in the worst case. (We could also define best case and average case time complexity functions, but here we concentrate on the worst case.)

Time complexity functions are a tool that allows us to estimate which of two algorithms we may prefer. We will examine the relationships between time complexity functions more rigorously in Section 5.

Objectives for Section 3

After studying this section you should be able to do the following.

- ☐ Recognise and use the terminology: time complexity function (of an implementation); sequential search; binary search.
- ☐ Count statement executions to determine the time complexity function of a given implementation (either iterative or recursive), in simple cases.

Exercises on Section 3

The exercises for this section both concern the function *FACTORIAL*, which was introduced in Unit 13. Recall from Section 1 that *FACTORIAL*(n) may be written as $n!$. See Unit 13, p. 6.

function *FACTORIAL*(n in Int) **return** in Int

pre $n > 0$.

post The returned value is formed by multiplying together all the integers between 1 and n .

Exercise 3.1

- (a) An implementation *itFact* of *FACTORIAL* is given below. For a general input integer n , determine how many times each of the statements in *itFact* will be executed. Hence determine a formula for $T_{itFact}(n)$, where n is the input integer.

```
function FACTORIAL(n)
{
    //itFact
    var i in Int           // (1)
    var result in Int      // (2)
    result <-- 1           // (3)
    for (i <-- 1 to n)      // (4)
    {
        result <-- result * i // (5)
    }
    return result          // (6)
}
```

- (b) What is the effect on the value returned if statement (4) in *itFact* is replaced by **for** ($i \leftarrow 2$ to n). What effect does this have on T_{itFact} ?

Exercise 3.2

A recursive implementation, *recFact*, of *FACTORIAL* is given below.

```
function FACTORIAL(n)
{
    // recFact
    var fac in Int           // (1)
    if (n == 1)             // (2)
    {
        fac <-- 1           // (3)
    }
    else
    {
        fac <-- n * FACTORIAL(n - 1) // (4)
    }
    return fac              // (5)
}
```

- (a) Follow the execution of *recFact* when the input n is 3. Show every statement that is executed, and so determine the total number of statement executions for this input.
- (b) Now consider a general input integer $n \geq 1$.
- How many statements are executed when $n = 1$ (the stopping case)?
 - For $n > 1$:
how many times will a recursive call be made;
how many statements are executed, other than those implicit in the pending recursive call?
 - Determine a formula for $T_{recFact}(n)$.

4 Recurrence systems

In Section 3, our approach to reasoning about the time taken to execute an implementation was to look for a pattern in how many statements are executed relative to the size of the input. In this section, we shall consider a different approach to determining time complexity functions. This method can be useful in more complicated examples, where the structure of an implementation makes the approach taken in Section 3 difficult to use.

The approach we describe here is applicable only to recursive implementations. It uses the recursive structure of the algorithm to obtain a corresponding recursive description of the time complexity function — by obtaining a recurrence system that the time complexity function must satisfy. However, knowing such a recurrence system takes us only part of the way to obtaining a useful expression for the time complexity function. We really need this expressed as a formula. So in the second part of this section, we shall look at ways of solving a given recurrence system — that is, of finding a formula equivalent to a function given by a recurrence system.

The size of the input was measured in various ways; for example, as the length of an input vector. In some cases it was simply the magnitude of an input integer.

You will see why we particularly want a formula when you have studied Section 5.

4.1 Recurrence systems for time complexity

Let us revisit the first recursive implementation we met in this unit: *recAddAll* implements the function *ADDALL*, which returns the sum of a sequence of integers stored in a vector. Here, the size of the problem, n , is the size of the vector. In Activity 3.5, we inferred that the time complexity function, $T_{\text{recAddAll}}(n) = 8n + 6$.

See Example 3.2.

```
function ADDALL(v)
{
    //recAddAll
    var temp in Vector of Int    // (1)
    var res in Int              // (2)
    temp <-- v.clone()          // (3)
    if (temp.size() == 0) then  // (4)
    {
        res <-- 0               // (5)
    }
    else
    {
        res <-- temp.last()     // (6)
        temp.removeLast()      // (7)
        res <-- res + ADDALL(temp) // (8)
    }
    return res                  // (9)
}
```

Now we can reason about the time complexity function $T_{\text{recAddAll}}(n)$ as follows. In the stopping case, when v is empty, the following statements are executed: (1), (2), (3), (4), (5) and (9), so there are a total of six statement executions. In this case, the size of the input is 0, so this gives us:

$$T_{\text{recAddAll}}(0) = 6. \quad (\text{S})$$

Now consider an input vector of size $n > 0$. In this case, the condition in (4) is **false**. The following statements are executed: (1), (2), (3), (4), (6), (7), (8) and, after the pending recursive call in (8) has been evaluated, (9). So the total number of statement executions is eight plus those executions required to find $ADDALL(temp)$ in (8). Now $temp$ will be a vector containing one less item than the input vector, and so the size of $temp$ is $n - 1$. Thus the number of statement executions required for the recursive call will be $T_{recAddAll}(n - 1)$. So we can see that:

$$T_{recAddAll}(n) = T_{recAddAll}(n - 1) + 8 \quad (n > 0). \quad (R)$$

Together, (S) and (R) form a recurrence system that $T_{recAddAll}$ must satisfy.

Since this recurrence system has a fairly straightforward form, we can see quite easily how we might obtain an equivalent formula. (R) states that (for $n > 0$) the number of statement executions is that required for an input of size one less, plus the eight statements required by the non-recursive part of the execution. Each recursive call reduces the size of the input by one, and adds eight to the total of statement executions forming $T_{recAddAll}(n)$. The recursive calls stop when the size of the input vector is 0, so there are n recursive calls. There are an additional six statement executions required by the stopping case. That gives $8n + 6$ calls in all. This is the reasoning we used in Section 3. However, it is not so easy to reason in this way if the implementation structure is more complicated, which will be reflected in a more complicated recurrence system.

Having obtained a recurrence system, and if we already have a formula for the time complexity function obtained by this sort of approach, it is useful to verify that the formula is indeed a solution of the recurrence system. We can do this using mathematical induction, as you saw in Unit 14.

At (3), v is copied to $temp$, and then one item is removed from $temp$ at (7)

Activity 4.1

Suppose that the function $T(n)$ satisfies the recurrence system below.

$$T(0) = 6 \quad (S)$$

$$T(n) = T(n - 1) + 8 \quad (n > 0) \quad (R)$$

Use mathematical induction to prove that $T(n) = 8n + 6$ is a solution of this recurrence system.

For convenience, we have dropped the subscript of $T_{recAddAll}$ here.

Discussion We lay out the inductive proof in the way used in Unit 14, Section 2. We begin by defining a separate function F giving the formula. Let $F(n) = 8n + 6$.

With $n = 0$, this formula gives $F(0) = 8 \times 0 + 6 = 6$. This is the same as the initial condition, $T(0) = 6$, of the recurrence system.

Now, for $k > 0$, suppose that the formula F gives $T(k - 1)$ correctly, so that

$$T(k - 1) = F(k - 1) = 8 \times (k - 1) + 6 = 8k - 2. \quad (H)$$

Then, for $k > 0$,

$$\begin{aligned} T(k) &= T(k - 1) + 8 && \text{(from the recurrence system)} \\ &= 8k - 2 + 8 && \text{(from H)} \\ &= 8k + 6 = F(k). \end{aligned}$$

Thus, for any integer $k > 0$, if the formula F gives $T(k - 1)$ correctly then it also gives $T(k)$ correctly. Having shown that the starting step and the inductive step hold, we can deduce, using the principle of mathematical

induction, that the formula $8n + 6$ gives the same values for all $n \geq 0$ as does the function $T(n)$ defined by the recurrence.

We now consider the implementation given below, of a function *STRREV* that reverses the order of the characters in a string. See Unit 13, p. 26.

```
function STRREV(s)
{
    // recStrRev
    var u in mString          // (1)
    var c in Char              // (2)
    if (s.length() == 1) then  // (3)
    { u <-- s }                // (4)
    else
    {
        u <-- s.substr(2,s.length() - 1) // (5)
        u <-- STRREV(u)                // (6)
        c <-- s.charAt(1)              // (7)
        u <-- u.charAppend(c)          // (8)
    }
    return u                    // (9)
}
```

Activity 4.2

Obtain a recurrence system satisfied by the time complexity function of *recStrRev*.

Discussion In this case, an appropriate measure of the size of the input is the length of the input string s . The stopping case is when s has length 1. In this case, statements (1), (2), (3), (4) and (9) are executed, so

$$T_{\text{recStrRev}}(1) = 5. \quad (\text{S})$$

If $n > 1$, then eight statements — (1), (2), (3) and (5)–(9) — are executed, together with those required by the pending recursive call in (6). This calls *recStrRev* for a string whose length, as a result of statement (5), is 1 less than that of s . So we have

$$T_{\text{recStrRev}}(n) = T_{\text{recStrRev}}(n - 1) + 8 \quad (n > 1). \quad (\text{R})$$

Together, (S) and (R) form the required recurrence system.

In the next example, we consider an implementation of the function *INTREE*, given below. Here, the input whose size determines the number of statement executions is a binary tree. To measure the size of a binary tree, we can use the depth of the tree. This specification does not require the input tree to be a BST, so in the implementation we need to check all the nodes to see whether x appears in t . To do this, we use recursive calls on both the left and right subtrees. See Unit 13, p. 37.

function *INTREE*(t in BinTree of X , x in X) **return** in Bool

pre true.

post The returned value is **true** if x appears at one or more nodes in t and is **false** otherwise.

```

function INTREE(t,x)
{
    //recInTree
    var b in Bool // (1)
    if (t.isEmpty()) then // (2)
        { b <-- false } // (3)
    else
    {
        if (x == t.getRoot()) then // (4)
            { b <-- true } // (5)
        else
        {
            b <-- INTREE(t.leftTree(),x)\INTREE(t.rightTree(),x) // (6)
        }
    }
    return b // (7)
}

```

In expressing the worst case for this implementation, recall that a binary tree is **full** if every level in the tree is completely filled, i.e. the tree contains the greatest number of nodes that is possible for a given depth. See Unit 14, p. 6.

Activity 4.3

Obtain a recurrence system satisfied by the time complexity function of *recInTree*.

Discussion The stopping case occurs when the tree is empty, that is, when its depth is 0. In that case, only statements (1), (2), (3) and (7) are executed. So

$$T_{\text{recInTree}}(0) = 4. \quad (\text{S})$$

If t has depth $n > 0$, we need to make sure that we are considering the “worst case” in determining the recurrence system for $T_{\text{recInTree}}(n)$. The worst case for this implementation will occur when: the item x does not appear in the tree t , and the tree t is full. In that case, statements (1), (2), (4), (6) and (7) will be executed (statement (7) will be executed after the pending recursive calls in (6)). Statement (6) involves two recursive calls — one on each of the subtrees of t . If t is full, then both its subtrees will be full, and each will have depth 1 less than the depth of t . So each recursive call will involve $T_{\text{recInTree}}(n - 1)$ statement executions. Thus

$$T_{\text{recInTree}}(n) = 2 \times T_{\text{recInTree}}(n - 1) + 5 \quad (n > 0). \quad (\text{R})$$

Together, (S) and (R) provide a recurrence system for the time complexity function $T_{\text{recInTree}}$.

4.2 Solving recurrence systems

Now suppose that we do not have an idea about a formula for the solution of a recurrence system. How might we find one? There are various mathematical techniques for solving certain forms of recurrence system, but here we shall consider just one approach — one that can help with the sort of recurrence system that may come up for time complexity functions. For now, we shall approach this as a purely mathematical task, without reference to the situation from which the recurrence system may have come.

We shall first consider how we might we solve the recurrence system below.

$$\begin{aligned} T(1) &= 5 & (S) \\ T(n) &= T(n-1) + 8 \quad (n > 1) & (R) \end{aligned}$$

The first point to note is that (R) can be used for any value of n that is larger than 1. For example, if n is 4, (R) can be used with n equal to each of 4, 3 and 2.

$$\begin{aligned} T(4) &= T(3) + 8 \\ T(3) &= T(2) + 8 \\ T(2) &= T(1) + 8 \end{aligned}$$

Now we have three equations here, so if we sum the left-hand sides and sum the right-hand sides, we will still have an equation. We get:

$$T(4) + T(3) + T(2) = T(3) + T(2) + T(1) + 3 \times 8.$$

Certain terms appear on both sides here ($T(3)$ and $T(2)$). We can subtract these from each side, to get:

$$T(4) = T(1) + 3 \times 8.$$

We know from (S) that $T(1)$ is 5, so this gives

$$T(4) = 5 + 3 \times 8 = 5 + 24 = 29.$$

Now our aim is to obtain a formula for $T(n)$ in terms of n . To do this, we can generalise the approach above. We need to start our list of equations with one for $T(n)$. We also include Equation (S) at the bottom of the list, as below.

$$\begin{aligned} T(n) &= T(n-1) + 8 \\ T(n-1) &= T(n-2) + 8 \\ &\vdots \\ T(3) &= T(2) + 8 \\ T(2) &= T(1) + 8 \\ T(1) &= 5 \end{aligned}$$

Notice that this list includes $n-1$ equations with an 8 on the right-hand side. If we sum each side and equate the results, we get:

$$\begin{aligned} T(n) + T(n-1) + \dots + T(3) + T(2) + T(1) \\ = T(n-1) + T(n-2) + \dots + T(2) + T(1) + (n-1) \times 8 + 5. \end{aligned}$$

Removing all the terms that appear on both sides, we get

$$T(n) = (n-1) \times 8 + 5 = 8n - 8 + 5 = 8n - 3.$$

This gives us a formula for $T(n)$ — a solution of the recurrence system for $T(n)$ given above. It is rather easy in such a calculation to make an algebraic slip. We can use mathematical induction (as in Activity 4.1) to confirm a formula obtained using this technique. (At a minimum, check the starting case! Here, the formula gives $T(1) = 8 - 3 = 5$, which corresponds with the value given for $T(1)$ in (S) in the recurrence system.)

Details of this are omitted.

Unfortunately, the summation of terms is not always as simple as in this example. Consider the problem of solving the recurrence system below.

See Activity 4.3.

$$\begin{aligned} T(0) &= 4 \\ T(n) &= 2 \times T(n-1) + 5 \end{aligned}$$

We might try to use an approach like that above. If we start in exactly the same way, we obtain the following.

$$\begin{aligned}
 T(n) &= 2T(n-1) + 5 \\
 T(n-1) &= 2T(n-2) + 5 \\
 &\vdots \\
 T(1) &= 2T(0) + 5 \\
 T(0) &= 4
 \end{aligned}$$

If we sum the left- and right-hand sides, and equate the results, we get:

$$\begin{aligned}
 T(n) + T(n-1) + \dots + T(1) + T(0) \\
 = 2T(n-1) + 5 + 2T(n-2) + 5 + \dots + 2T(0) + 5 + 4.
 \end{aligned}$$

With this equation, we are not able to remove the terms we do not want: $T(n-1)$, $T(n-2)$, and so on. That is because we have $T(n-1)$ on one side of the equation, but have $2T(n-1)$ on the other side. To make this sort of approach work, we need to do something more subtle. To have any chance, we need the second equation to give an expression for $2T(n-1)$. So to start, we will multiply the second equation above by 2. That gives

$$\begin{aligned}
 T(n) &= 2T(n-1) + 5 \\
 2T(n-1) &= 2(2T(n-2) + 5) = 2^2T(n-2) + 2 \times 5
 \end{aligned}$$

If the term in $T(n-2)$ is going to disappear when we add up all the equations, we now need to multiply the third equation by 2^2 , and we need to repeat this pattern all the way through.

Remember, to still have an equation we must multiply every term in the original equation by the same quantity, so we multiply every term in the equation by 2.

EXAMPLE 4.1

To solve the recurrence system

$$\begin{aligned}
 T(0) &= 4 \\
 T(n) &= 2 \times T(n-1) + 5
 \end{aligned}$$

we can proceed as below.

$$\begin{aligned}
 T(n) &= 2T(n-1) + 5 & = 2T(n-1) + 5 \\
 2T(n-1) &= 2(2T(n-2) + 5) & = 2^2T(n-2) + 2^1 \times 5 \\
 2^2T(n-2) &= 2^2(2T(n-3) + 5) & = 2^3T(n-3) + 2^2 \times 5 \\
 2^3T(n-3) &= 2^3(2T(n-4) + 5) & = 2^4T(n-4) + 2^3 \times 5 \\
 &\vdots \\
 2^{n-1}T(1) &= 2^{n-1}(2T(0) + 5) & = 2^nT(0) + 2^{n-1} \times 5 \\
 2^nT(0) & & = 2^n \times 4
 \end{aligned}$$

Each equation is multiplied by an additional factor of 2. The second equation is multiplied by 2, the third equation by 2^2 , the fourth equation by 2^3 , and so on.

The first column can be added up to give the sum of the left-hand sides of each equation, and the third column can be added up to give the sum of the right-hand sides of each equation. Notice that the term $2T(n-1)$ appears on both sides, so we can subtract it from both sums and so remove it. Likewise we can subtract the remaining terms in the first column from both sides, to leave just $T(n)$ on the left-hand side. This leaves us with:

$$T(n) = 5 + 2^1 \times 5 + 2^2 \times 5 + \dots + 2^{n-1} \times 5 + 2^n \times 4.$$

Most of the terms on the right-hand side contain a factor of 5, so:

$$T(n) = 5(1 + 2 + 2^2 + \dots + 2^{n-1}) + 4 \times 2^n.$$

To evaluate the sum in this equation we can use a formula given in Unit 14. See Unit 14, Result 2.3.

$$1 + 2 + 2^2 + \dots + 2^k = (2^{k+1} - 1)/(2 - 1) = 2^{k+1} - 1.$$

Using this, with k equal to $n-1$, we obtain:

$$T(n) = 5(2^{n-1+1} - 1) + 4 \times 2^n = 5(2^n - 1) + 4 \times 2^n = 9 \times 2^n - 5.$$

It is rather easy to make a slip in the sort of calculation given in Example 4.1. So it is advisable to use proof by induction to confirm that the expression one has arrived at really is a solution of the recurrence system. As a quick check, note that with $n = 0$ the formula gives $9 \times 2^0 - 5 = 9 - 5 = 4$, which is the value given for $T(0)$ in the recurrence system. The term 2^n in the solution arises from the fact that, in the recurrence system, $T(n)$ depends on $2 \times T(n - 1)$ (rather than just $T(n - 1)$).

Details are omitted.

Solutions to recurrence systems often rely on knowledge of a formula, such as the one above, to simplify a summation, and the equations may have to be manipulated, as we did in this example, in order to get terms to cancel after we sum our equations.

You may not be comfortable with the sort of algebra needed for this example. Since you have nearly reached the end of the course, you will know that such algebra does not play an important role in your work!

The recurrence system in Example 4.1 was obtained in Activity 4.3 for the time complexity function of the implementation *recInTree*. The appearance of a term of the form 2^n in the formula for a time complexity function would usually indicate that the implementation is not of practical use for a large input. However, we need to be a little cautious in how we interpret such a time complexity function where there is a tree input whose size is measured as the depth of the tree. Recall that a full tree of depth n contains $2^n - 1$ nodes. For a tree input, one needs to consider whether “the problem is large” means that “the depth of the tree is large” or “the number of nodes in the tree is large”. Informally, *recInTree* is similar to *seqSearch*: in each case, we search through a collection of items in a way that may require us to examine every item.

It is beyond the scope of this unit to explore further the task of solving recurrence systems. There are many textbooks on discrete mathematics for computer science that you can consult, should you be interested in the topic or should the need arise.

Objectives for Section 4

After studying this section you should be able to do the following.

- ☐ Identify a recurrence system governing the time complexity function of a simple recursive algorithm.
- ☐ Solve a recurrence system of a suitable form.
- ☐ Use mathematical induction to confirm that a given formula is a solution of a recurrence system.

Exercises on Section 4

Exercise 4.1

In Exercise 3.2, a recursive algorithm *recFact* was given

- (a) Give a recurrence system for the time complexity function of *recFact*.
- (b) Use mathematical induction to confirm that the formula $4n$ is a solution of the recurrence system you gave in part (a).

See Solution 3.2.

Exercise 4.2

Solve the recurrence system below.

$$\begin{aligned}T(n) &= T(n - 1) + 2n & (n > 1) \\T(1) &= 1\end{aligned}$$

You will need to use the fact that $1 + 2 + \dots + n = n(n + 1)/2$.

See Unit 14, Result 2.3.

5 Comparison of time complexity functions

We want to be able to compare the times required to execute alternative implementations as the problem size grows. To do this, we adopt a commonly used classification scheme and a notation called **big Oh**.

Our major concern is what happens when we deal with large inputs because, amongst other things, testing an algorithm on small inputs may provide running times that give a false sense of feasibility. The behaviour of a function, $f(n)$, for large values of n is called its **asymptotic behaviour**. A formal definition is given in Subsection 5.2, where we shall see how we can use the formula of a time complexity function to rapidly classify its asymptotic behaviour. First, we shall take a less formal look at functions that may arise in time complexity.

5.1 Comparing graphs

We have already encountered time complexity functions such as $T(n) = 4n + 8$ and $T(n) = 8\log_2(n + 1) + 5$. In some cases, we drew up tables of values showing the values of $T(n)$ for various values of n .

The following table shows some functions that often arise in time complexity, and their values for selected values of n . Some rounding has been done. It will come as no surprise that some functions grow faster than others.

Input size	Time complexity function $T(n)$.						
n	1	$\log_2(n)$	n	$n\log_2(n)$	n^2	2^n	$n!$
5	1	2.3	5	11.6	25	32	120
10	1	3.3	10	33.2	100	1024	3.6×10^6
100	1	6.6	100	664.4	10 000	1.3×10^{30}	9.3×10^{157}
1000	1	10.0	1000	9965.8	1×10^6	1.1×10^{301}	<i>Inf</i>
10 000	1	13.3	10 000	1.3×10^5	1×10^8	<i>Inf</i>	<i>Inf</i>

The entries marked *Inf* reflect an overflow in the program used to calculate the values; in other words, it represents a number larger than can be stored by the data type used to do the calculation. For a 64-bit floating-point type, this occurs for values larger than approximately 1.7×10^{308} . This is the largest floating-point number that can be stored in 64 bits, under the IEEE 754 arithmetic standard.

How does this relate to the time required to run a program of the given time complexity?

A 1 GHz processor is capable of executing one billion (10^9) instructions per second. The processor takes 10^{-9} seconds (one nanosecond, written 1 ns) to perform one instruction. For illustration, let us assume that the figures in the table above represent how many instructions have to be executed by our processor, and that the processor is just dealing with our program. Then the table below gives the corresponding running times for a 1 GHz processor.

In IEEE 754 arithmetic, *Inf* is a value that can be used in arithmetic operations. It acts as a signal of an overflow, but does not cause a program crash. (The IEEE 754 arithmetic standard was developed by the Institute of Electrical and Electronics Engineers. It is used by many programming languages.)

Input size	Running time for time complexity function $T(n)$.						
n	1	$\log_2(n)$	n	$n \log_2(n)$	n^2	2^n	$n!$
5	1 ns	2.3 ns	5 ns	12 ns	25 ns	32 ns	120 ns
10	1 ns	3.3 ns	10 ns	33 ns	100 ns	0.001 ms	3.6 ms
100	1 ns	6.6 ns	100 ns	700 ns	0.01 ms	4×10^{13}	*
						years	
1000	1 ns	10 ns	0.001 ms	0.01 ms	1 ms	*	*
10 000	1 ns	13 ns	0.01 ms	0.13 ms	100 ms	*	*

1 ns = 10^{-9} seconds, a nanosecond.

1 ms = 10^{-3} seconds, a millisecond.

The contrast is stark. For example, $T(n) = n^2$ with n equal to 10 000 requires one-tenth of a second, while $T(n) = 2^n$ with n equal to 100 is so large that you would have to wait 40 billion millennia for your program to stop running. This is longer than the universe is reckoned to have been in existence, by a long way! (A current estimate of the age of the universe is 13 billion years, about three thousand times less than such a program needs to complete.)

The times marked * are so large that there is no human measure of time that makes any sense of them. They are much longer than billions of millennia. It is not worth considering running a program for these lengths of time — nor for forty billion millennia!

This table shows that, for some inputs, some programs take so long to run that they are not worth running. (This does not mean that the algorithm is difficult; it may be very simple, but require lots of computing time to complete running.) If all known algorithms to solve a problem perform badly for relatively large n , we say the problem is **computationally hard**.

Activity 5.1

Assume that an average computer user is not willing to wait more than one second for a calculation to be completed, and that the processor is capable of 10^9 operations per second. If $T(n)$ gives the number of operations required by a particular calculation, what is the largest problem that can be solved when $T(n)$ is each of the following?

- (a) $T(n) = \log_2(n)$; (b) $T(n) = n$; (c) $T(n) = n^2$; (d) $T(n) = 2^n$.

Discussion In each case, we need to find the value of n at which $T(n)$ becomes 10^9 .

- (a) For what value of n is $\log_2(n) = 10^9$? To solve this, we can use the fact that, if $x = \log_2(n)$, then $n = 2^x$. So we find that $n = 2^{1000\,000\,000}$. (In fact, this number is larger than is provided for by standard computer arithmetic, so we would encounter problems with computer arithmetic before we had to worry about the algorithm's running time.) In other words, with such an algorithm running time would not be an issue for extremely large problem sizes — in theory.
- (b) For $T(n) = n$, the question is: when is $n = 10^9$? Not too hard, this one: the answer is clearly $n = 10^9$, or 1000 000 000 (one billion).
- (c) For $T(n) = n^2$ we need to solve $n^2 = 10^9$. Taking the square root of each side of the equation, we obtain $n = 10^{4.5}$, which is more than 10^4 (10 000), but less than 10^5 (100 000). (To be exact, $10^{4.5} = 10^4 \times \sqrt{10} = 31\,620$.)

- (d) For $T(n) = 2^n$ we want $2^n = 10^9$. We can take logs to base two of each side of the equation to find $n = \log_2(10^9) = 9\log_2(10)$. Now $\log_2(10)$ is that power of 2 that gives 10; because $2^3 = 8$ and $2^4 = 16$, we can see that $\log_2(10)$ is between three and four. So $9\log_2(10)$ is more than $9 \times 3 = 27$, but less than $9 \times 4 = 36$. (You can use a calculator to find $\log_2(10)$ exactly, and so find the largest possible integer problem size.)

The last result in the above discussion is particularly interesting, since it tells us that if our time complexity function is exponential, we cannot even handle a problem size of 36.

You can guess that a time complexity function involving a factorial would give you an even smaller tolerable problem size. (You can work it out by trial and error with a calculator.)

To help in visualising how a function $T(n)$ grows as n increases, we can display values of $T(n)$ in a graph. We shall start by looking at the graphs of some simple functions — the “building blocks” from which a formula for $T(n)$ may be formed, such as n , $\log_2(n)$ or n^2 .

In these graphs, we are really only interested in values of n that are integers with $n \geq 0$. A negative problem size is unlikely to be meaningful; nor is a problem size such as 2.5. However, in showing the graphs it is conventional to draw continuous lines (values for all positive numbers n), and this can make visual comparison of graphs easier.

We look first at expressions such as $5 + 3n + 2n^2 + 6n^3$. This expression is an example of a **polynomial** in n . The general form of a polynomial is $T(n) = a_1n^c + a_2n^{c-1} + \dots + a_{c-1}n + a_c$, where c is a positive integer, and the values a_i are all fixed numbers (that is, they do not depend on n). In a polynomial, the integer c is called the **degree** of the polynomial. A polynomial of degree 1 (such as $8n + 5$) is known as **linear**, a polynomial of degree 2 is called **quadratic**, and a polynomial of degree 3 is called **cubic**. We also regard a function with $c = 0$ as a special form of polynomial. In that case we have a **constant** function — one whose value does not depend on n . (Naturally, we are very interested in any implementation whose time complexity function has this property.)

The graphs in Figure 5.1 show some of the basic terms from which a general polynomial is built: n (linear), n^2 (quadratic) and n^3 (cubic), as well as the graph of 1 (constant). We are interested in the way the value of $T(n)$ increases as n increases, which will be shown by the way the graph moves upwards as you look from left to right. Notice that the rate of increase becomes greater as we move from linear to quadratic to cubic.

These graphs will become familiar to you as we explore them further.

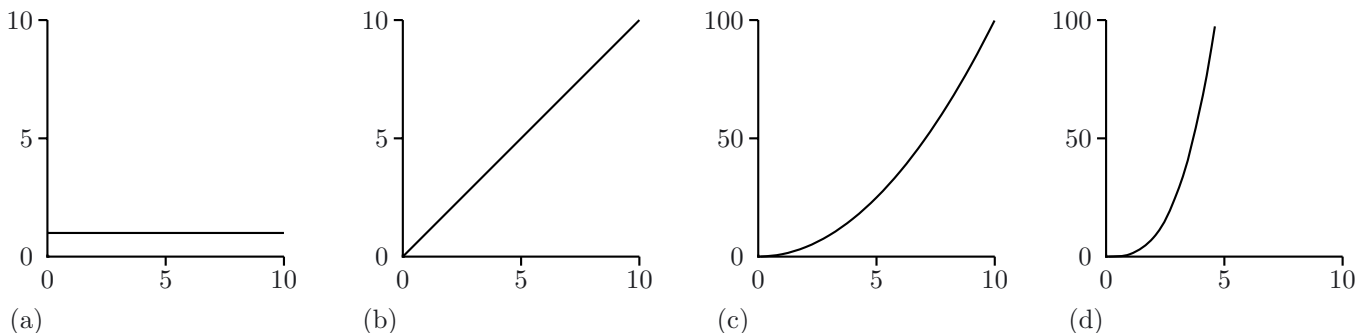


Figure 5.1 Graphs of polynomial functions: (a) 1; (b) n ; (c) n^2 ; (d) n^3 .

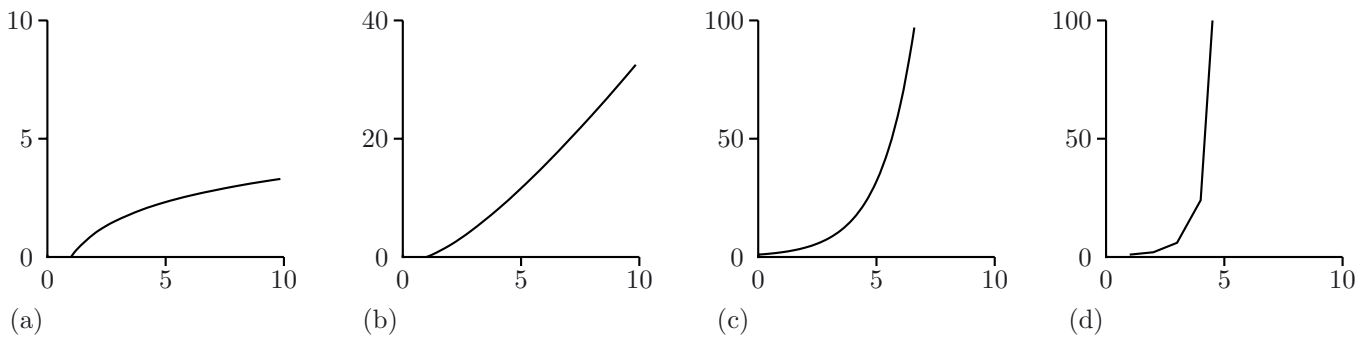


Figure 5.2 Graphs of: (a) $\log_2(n)$; (b) $n \log_2(n)$; (c) 2^n ; (d) $n!$.

Other functions that may arise in time complexity are logarithms, exponentials (such as 2^n), and factorials — see Figure 5.2. (Values are shown for $n \geq 1$, since $\log_2(0)$ is not defined. We have defined $n!$ only for integer values of n — in the graph, these points are joined by straight lines.)

The question now arises as to which functions are more desirable, in terms of time efficiency. We can get a feeling for this by showing several graphs in a single diagram. In Figure 5.3, we compare some sample polynomial functions. These graphs suggest that the larger c is, the faster n^c increases. Notice that constant multiples have relatively little effect on the behaviour as n increases. For example, n^2 is less than $3n$ for $n < 3$, but once the graphs cross at $n = 3$, then n^2 gets bigger than $3n$ at an ever-increasing rate.

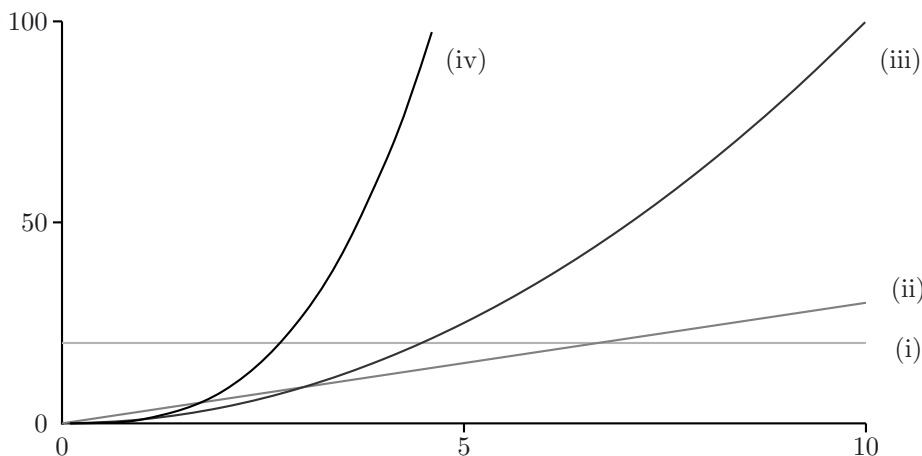


Figure 5.3 Comparing polynomials. Graphs of: (i) 20; (ii) $3n$; (iii) n^2 ; (iv) n^3 .

The relationship between logarithmic functions and polynomials is more subtle (see Figure 5.4(a)). As n increases, the function $\log_2(n)$ grows faster than a constant, but more slowly than a linear polynomial, such as $2n$. The function $n \log_2(n)$ grows more quickly than a linear function, but more slowly than a quadratic such as n^2 . It is not surprising that $n \log_2(n)$ grows faster than n , because it multiplies it by a function — $\log_2(n)$ — which is itself growing. Likewise we would expect $n \log_2(n)$ to grow more slowly than n^2 , since $\log_2(n)$ grows more slowly than n .

Exponential and factorial functions grow very quickly. They soon overtake the quadratic function, as illustrated in Figure 5.4(b). (Indeed, they will overtake any polynomial eventually, however large the degree of polynomial.) Notice that when n is small, an exponential or factorial may be smaller than a polynomial.

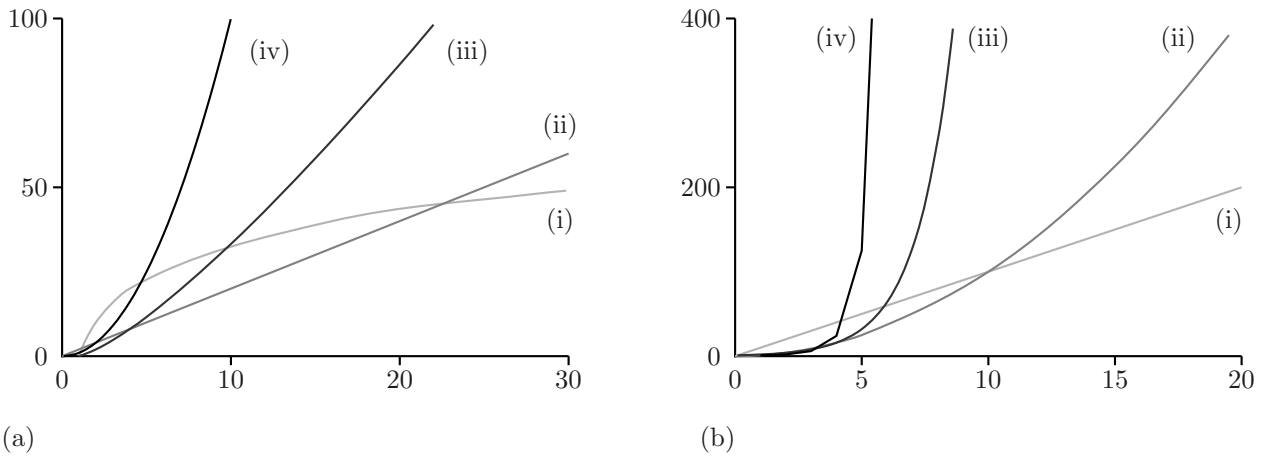


Figure 5.4 Comparing graphs.

(a) (i) $10 \log_2(n)$; (ii) $2n$; (iii) $n \log_2(n)$; (iv) n^2 .

(b) (i) $10n$; (ii) n^2 ; (iii) 2^n ; (iv) $n!$.

We have only looked at a few functions, but these turn out to be a good representative sample. The functions that we have examined are representatives of classes of function. We shall develop these examples into a more rigorous description of efficiency in the next subsection. However, the intuitions just gained from examining graphs will not, in general, mislead us.

Activity 5.2

- Sketch a graph of $f(n) = 2n^2$ for values of n in increments of 10 from 0 to 100. On the same axes, sketch a graph of $g(n) = 100n$.
- When is $f(n)$: less than $g(n)$; equal to $g(n)$; greater than $g(n)$?
- Based on your answer to (b), what would you say about algorithms with time complexities given by f and g respectively?

Discussion

- Figure 5.5 shows graphs of: (i) $2n^2$ and (ii) $100n$. (These can be produced using a graphical calculator, by tabulating then plotting values, or by sketching the graphs by hand — whichever you find convenient.)
- We can solve this problem directly. Consider first the situation when $2n^2$ is smaller than $100n$. For this to be the case, n must satisfy the inequality $2n^2 < 100n$. Since n is positive, we can divide both sides of this inequality by n , to get $2n < 100$, or $n < 50$. A similar approach shows us that the two are equal when $n = 50$, and that the quadratic function, f , is larger when $n > 50$.
- We note that the quadratic becomes larger than the linear function when n is large enough. If we know that the problem sizes will be greater than 50 most of the time, this would suggest that the algorithm with linear time complexity is preferable. If the problem sizes are less than 50, the algorithm with quadratic time complexity may be preferable. However, we need to be cautious when drawing conclusions from time complexity functions — see the comments at the end of this section. For now, bear in mind that our time complexity functions are based on a number of simplifying assumptions.

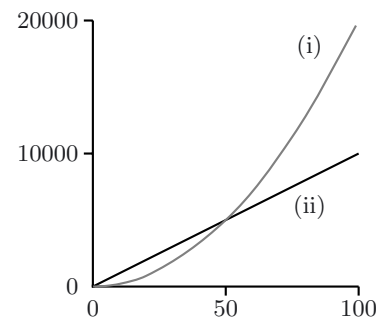


Figure 5.5 (i) $2n^2$; (ii) $100n$.

Activity 5.3

Use the graphs in Figure 5.4(a) to estimate when $2n$ becomes larger than $10 \log_2(n)$, and confirm your impression by calculating these expressions for a few values of n .

Discussion The graphs in Figure 5.4(a) (i) and (ii) appear to cross at about $n = 23$, after which $2n$ is larger. With $n = 23$, $10 \log_2(23) = 45.2$, which is already less than $2 \times 23 = 46$, while with $n = 22$, $10 \log_2(22) = 44.6$, which is greater than $2 \times 22 = 44$.

5.2 Big Oh notation

We want to be able to compare the behaviour of different functions $f(n)$ as n becomes large. We saw in the previous subsection that certain functions grow faster than others and that, to a large extent, this depends on a broad classification of the function rather on details of a formula. For example, we would expect n^2 to grow faster than $3n \log_2(n) + 5n + 11$, because n^2 grows faster than any of the basic terms involved in the second formula: $n \log_2(n)$, n or a constant.

To formalise this intuition, we can use the big Oh classification. This is based on the **asymptotic behaviour** of a function; that is, its behaviour for large values of n . A formal definition of this is given below.

For functions f and g , we say that g **asymptotically dominates** f if there are positive numbers c and N such that $f(n) \leq c \times g(n)$ for all $n \geq N$.

We will assume that the functions f and g concerned have positive values — this simplifies the definition slightly, and will be true of any time complexity function.

EXAMPLE 5.1

Consider the functions $f(n) = 3n$ and $g(n) = n^2$. We know (from the graphs) that g grows faster than f . Can we say that the quadratic, g , asymptotically dominates the linear polynomial, f ?

If g asymptotically dominates f , we must be able to find constants c and N satisfying the relation $3n \leq c \times n^2$ for all $n \geq N$.

The graph in Figure 5.3 suggests that if n is more than about 3, then n^2 will be more than $3n$. Suppose we choose $c = 1$ and $N = 3$; then the condition required in the definition of asymptotic domination becomes

$$3n \leq n^2 \text{ for all } n \geq 3.$$

This is true. Suppose that $n \geq 3$. Then we can multiply both sides of this inequality by the positive number n to deduce that $n^2 \geq 3n$. Therefore g asymptotically dominates f .

There will be other values of c and N that will also satisfy the required condition.

Activity 5.4

Let $f(n) = n$ and $g(n) = 3n$.

- (a) Does g asymptotically dominate f ?
- (b) Does f asymptotically dominate g ?

Discussion

- (a) It is easy to see that we have $n \leq 3n$ for all $n \geq 0$. So, choosing $N = 0$ and $c = 1$, we can say that g asymptotically dominates f .
- (b) We also have $3n \leq 3 \times n$ for all $n \geq 0$. So, choosing $N = 0$ and $c = 3$, we can say that f asymptotically dominates g as well!

Again, there will be other values of N and c that allow us to draw the same conclusions.

From this you should be able to see that asymptotic domination is not simply about whether one function is growing faster than another: it is more about comparing the ways in which functions grow. The two functions n and $3n$ dominate each other — and this suggests that they are growing “in the same way” (in a broad sense).

Activity 5.5

Consider $f(n) = 3n$ and $g(n) = n^2$. Can you find values of N and c to show that f dominates g ?

Discussion Choose a value of c , for example $c = 100$, and then examine the relation $n^2 \leq 100 \times 3n$. We see that this relation only holds if $n \leq 300$. There is no chance of finding a value of N such that $n^2 \leq 3n$ holds for all values of $n \geq N$ — the relation goes wrong as soon as n reaches 300.

Replacing 100 by a larger value for c would only postpone the point at which n^2 gets larger than $3n$. So f does not dominate g .

We saw in Example 5.1 that g dominates f .

The most commonly used approach to comparing growth rates is called **big Oh** notation. We now give the definition of this.

A function $f(n)$ is $O(g(n))$ if g asymptotically dominates f .

Read this as “ f is big Oh of g ”.

To say that $f(n)$ is $O(g(n))$ means that $f(n)$ grows no faster than $g(n)$ does. The function g provides an upper bound on how fast f is growing. It does not tell us that f is growing as fast as g is, only that it is not growing faster. For example, $f(n) = 3n$ is $O(n^2)$. However, this is not the most useful information we can give about the behaviour of the function $f(n) = 3n$ as n becomes large; we can also say that it is $O(n)$, as we saw in Activity 5.4.

An important feature to emphasise in the definitions of O and asymptotic domination is that these relate only to the behaviour of a function $f(n)$ *when n is large*. In the formal definition that $g(n)$ asymptotically dominates $f(n)$, we only require that a multiple of $g(n)$ becomes larger than $f(n)$ for values of n greater than (or equal to) some particular value N — and N could be very large. This is a point to bear in mind when interpreting information about a time complexity function. If we know that $T(n)$ is $O(g(n))$ for a time complexity function T , we know only that the time to execute the implementation grows as fast as g *when the input size is large enough*.

Activity 5.6

Suppose you are told (about time complexity functions T_1 and T_2) that $T_1(n)$ is $O(n)$ and $T_2(n)$ is $O(10n)$. What conclusions can you draw from this information?

Discussion We know that T_1 is asymptotically dominated by n and, looking back at the definition, this means that there are positive constants c and N such that $T_1(n) \leq c \times n$ for all $n \geq N$. Similarly, we know that T_2 is asymptotically dominated by $10n$, so there are positive constants d and M such that $T_2(n) \leq d \times 10n$ for all $n \geq M$.

The constants c and d can be any positive values, so there is no difference in the information we have about the functions $T_1(n)$ and $T_2(n)$. In each case, we know that their values do not grow faster than some linear function — say $a \times n$ (where a is a positive constant) — for large values of n .

As indicated by the above example, there is no difference in saying that a function is $O(n)$ or $O(10n)$, so we shall confine our classification of time complexity function to certain “standard” functions (and not include constant multiples).

An important limitation on the information provided by a big Oh classification is that this only provides an upper bound on the behaviour of a given time complexity function. For example, $T(n) = 3n$ is $O(n^2)$ but it is also $O(n)$, and this second piece of information is much more useful.

A measure that fits our needs here is big Theta. If we say that $f(n)$ is $\Theta(g(n))$, we mean that f grows about as fast as g . This is like having an upper and a lower bound that is the same function.

The upper bound, big Oh, is the measure most commonly seen, but here we shall concentrate on big Theta, which is the technically correct measure if we want to give the most useful information about the behaviour of a function $T(n)$ for large values of n . We shall refer to the classification of a function $T(n)$ using Θ as the **order** of $T(n)$. For example, we say that $T(n) = 3n$ is of order n (meaning that $3n$ is $\Theta(n)$).

In Table 5.1 below, we give the order of a variety of functions. The orders more commonly encountered in time complexity include $\log n$, n , $n \log n$ and n^2 . We have included a number of other functions in the table, which provides a ranking of efficiency for time complexity functions. The functions become less efficient as we move down the table.

If f is:	with conditions	we call it	Example
$\Theta(1)$	integer $c \geq 2$ $0 < r < 1$	constant	125
$\Theta(\log n)$		logarithmic	$10 \log n + 2$
$\Theta(\log^c n)$		polylogarithmic	$\log^2 n (= \log n \times \log n)$
$\Theta(n^r)$		sublinear	$n^{0.5} (= \sqrt{n})$
$\Theta(n)$	$1 < r < 2$	linear	$5n - 3$
$\Theta(n \log n)$		$(n \log n)$	$12n \log n + 4n - 6$
$\Theta(n^r)$		subquadratic	$n^{3/2} (= n\sqrt{n})$
$\Theta(n^2)$		quadratic	$2n^2 + 10n - 9$
$\Theta(n^3)$	integer $c \geq 4$	cubic	$7n^3 - 4$
$\Theta(n^c)$		polynomial	n^{100}
$\Theta(r^n)$		exponential	2^n
$\Theta(n!)$		factorial	$n!$

Table 5.1 A hierarchy of orders of functions, given by Θ .

The symbol Θ is a Greek capital theta, and can be read as “big theta”.

The base of any logarithm is not significant, because, for example, $\log_{10}(n)$ is a constant multiple of $\log_2(n)$.

We shall not use terms such as “polylogarithmic” — they are mentioned here in case you see them elsewhere.

An algorithm of exponential or factorial order would not normally be practicable to use for large values of n .

We summarise the orders that occur more often in time complexity in Table 5.2 below. The further to the right in this table that we find the order of the time complexity function, the less efficient the corresponding implementation is — in the *worst case*, and for *large inputs*.

$\Theta(1)$	$\Theta(\log n)$	$\Theta(n)$	$\Theta(n \log n)$	$\Theta(n^2)$	$\Theta(n^3)$	$\Theta(2^n)$
-------------	------------------	-------------	--------------------	---------------	---------------	---------------

Table 5.2 A hierarchy of commonly occurring time complexities

This summary list misses out functions that may occur; refer to Table 1 for other functions, or use “common sense” — for example a function of order $\Theta(n^5 \log n)$ is worse than one of order $\Theta(n^5)$.

Activity 5.7

- (a) If $f(n) = n^2$, is $f(n) = O(n^4)$?
- (b) What order is $g(n) = 2n^3 + n^2$?

Discussion

- (a) Yes, n^2 is $O(n^4)$. For example, choosing $c = 1$ and $N = 1$, we see that $n^2 \leq n^4$ for $n \geq 1$. This illustrates a pitfall of big Oh: it provides only an upper bound. If someone asked you how far it was to Coventry from Milton Keynes and you said “less than a million miles”, it would provide less information than if you said “less than a hundred miles”, though both might be true. We always try to provide the “tightest” upper bound we can, because it conveys the most information.
- (b) It is cubic, $\Theta(n^3)$. We can show that it is $O(n^3)$ by choosing $c = 10$ and $N = 1$, for we see that $2n^3 + n^2 \leq 10n^3$ for $n \geq 1$. Since the formula $2n^3 + n^2$ includes a term involving n^3 , there is no chance of supplying a “tighter” upper bound.

There are many other values of c and N that also satisfy the definition.

You have probably noticed, from the examples given in Table 5.1, and from Activity 5.7, that we can arrive at the order of a function given as a formula fairly quickly — so long as the terms in the formula appear in Table 5.1. We just ignore constant multiples, and look for the “worst” term in the formula. Here is a general technique to help you classify the order of a function

Classifying functions in general

1. Look for the term in the function that is furthest down the hierarchy in Table 5.1 above. This term will tend to dominate the value of the function when n is large.
2. Remove any multiplying factor on the dominant term — say the result is $g(n)$. For example, $2n^3$ becomes just n^3 .
3. The function is of order $\Theta(g(n))$.

For example, $8n$ is $\Theta(n)$, while $3n^4 + 2n^3 + 100$ is $\Theta(n^4)$

Activity 5.8

Give the order of each of the following functions.

- (a) $\log n + 2n^2 + 2$
- (b) $n \log n + 20 \log n$
- (c) $2^{10} + n$
- (d) 2^{10}

Discussion Remember that 2^{10} is just a constant — albeit a large one!

- (a) $\log n + 2n^2 + 2$ is $\Theta(n^2)$.
- (b) $n \log n + 20 \log n$ is $\Theta(n \log n)$.
- (c) $2^{10} + n$ is $\Theta(n)$.
- (d) 2^{10} is $\Theta(1)$.

Notice that, for example, the functions $2^{10} + n$ and $8n$ have the same order — they are “the same” as far as the Θ classification is concerned. If these were the time complexities of two implementations, then we would regard these implementation as more or less equally good, as measured by their order. When the problem size is large enough, there should not be much to choose between such implementations. However, as we explain next, we should make such statements with caution. Here are some points to bear in mind when comparing the efficiency of implementations by looking at the order of their time complexity functions.

Being careful with classification by order

1. If you are comparing time complexity functions and you know that the problem size will be small, big Oh — and so their order — is not a reliable way to compare them. (This is because for $f(n)$ to be $O(g(n))$, $g(n)$ only needs to dominate $f(n)$ for all $n \geq N$. The value N may be large.)
2. Remember that constant multiples are ignored in determining the order. This adds difficulty when interpreting its meaning. Two functions may be the same order, but the constant multiplier could be small in one case and huge in another. So we need to be cautious about saying that two algorithms of the same order are equivalent. The constants may dominate the running times observed. However, normally there is no reason to expect that the constant multipliers would be significantly different for the functions you are comparing.
3. We made simplifying assumptions when counting operations to find time complexity. There are various conditions that would make these assumptions invalid. One example is where a statement in a program invokes another function. We cannot tell how long the other function will take, but it is likely to be relatively long, and could dominate the running time of the code calling it. Another example is where code must interact with a hard disk, because disk operations are typically much slower than operations taking place in a computer’s main memory. So, we need to bear in mind that certain conditions can invalidate our assumptions.
4. To optimise program performance for small input sizes, it may be more useful to analyse which parts of the program are taking longest, and then concentrate on speeding up those parts of the program by fine tuning the implementation, or using assembly language.

A constant may be large because an algorithm is complex, or because some operations turn out to be very time consuming.

Objectives for Section 5

After studying this section you should be able to do the following.

- Recognise and use the terminology and notation: order of a function; asymptotic domination; $O(f(n))$, $\Theta(f(n))$.
- Determine the order of a function (using Table 5.1).
- Discuss the implications of a time complexity function being of a particular order.

Exercises on Section 5

Exercise 5.1

- (a) Given that $g(n) = n^2$ and $f(n)$ is $O(g(n))$, what can you say about f ?
- (b) If we know that $T(n) = 2^{100}$, what order is it?
- (c) Is $f(n) = 4n^3 + 3$ of order $O(n^5)$? Is this the best information we can give about f ?
- (d) Explain why knowing that one time complexity function is $\Theta(n^2)$ and another is $\Theta(n^3)$ does not of itself guarantee that the first is better than the second, even for large inputs.

Exercise 5.2

The time complexity functions for two different algorithms A and B to implement the same function are given by:

for A: $f(n) = 3n \log_2(n) + 10n$;

for B: $g(n) = 2n^2 + 4n$.

By classifying each of f and g as $\Theta(F(n))$ for a suitable function F , say which of the two algorithms is likely to be the more efficient for large n .

6 Postscripts

1. On counting statements in algorithms

We made a number of suppositions about the time taken to execute statements, including that they all take about the same time. Of course, this is not true. Conventionally, it is recognised that statements that involve comparisons ($<$, $>$, $==$, $<=$, $>=$), be they between numbers or between strings, generally take significantly longer than those that make assignments to variables or perform arithmetic operations. Many texts that examine complexity ignore, for accounting purposes, all code statements that do not involve comparisons. In some cases, this extends to ignoring those comparisons that do not take much time, such as those concerned with counting the iterations of a **for** loop. Such simplifications will usually have no significant bearing on the outcome of complexity analysis as we have visited it in this unit. The complexity functions obtained by different approaches will differ in their detail, depending upon the simplifications made, but in general they will have the same order.

2. On space complexity

Just as we have analysed complexity in terms of the time required by an algorithm, so it is also possible to analyse the space (memory) required by an algorithm. We have already alluded to the fact that recursive algorithms must keep a record of their pending computations before making a recursive call. You can readily imagine that this requires more and more memory or storage as the depth of recursion grows. Just as we saw that certain problems require times that are well beyond our best estimates of the lifetime of the universe, so it is that analysis of space complexity reveals that many everyday problems require more matter than is available in the universe, even if we were to use single subatomic particles as the basic storage elements! We do not expect you to know about the details of space complexity, just to be aware of it.

3. On ‘eyeballing’ code

Computer programmers tend to quickly assess the complexity of portions of code, following simple rules like the following.

- a. If code has no loops, and no recursion, it may well be $\Theta(1)$.
- b. If code proceeds by repeatedly halving the size of a problem (“divide and conquer”), it may well be $\Theta(\log n)$.
- c. If code has a single loop as part of its main work (and the frequency with which the loop is executed depends on the input size), it may well be $\Theta(n)$.
- d. If code has a nested loop as part of its main work, it may well be $\Theta(n^2)$.

Although these are only rules of thumb, they are useful in assisting a programmer to quickly spot code that may be inefficient, or efficient.

4. Algorithms and implementations

In this unit, we have concentrated on analysing implementations — where we can examine the code in detail. An algorithm is a general approach to the solution of a problem. We would, though, expect the description of an algorithm to be sufficiently detailed to mean that any implementation of it will have the same order.

5. Revisiting the supermarket delivery problem.

The supermarket delivery problem, introduced in Section 1, is an example of what is generally known as the Travelling Salesman Problem (TSP). Informally stated, our algorithm depended upon a strategy of considering each and every one of the possible alternative routes, computing the length of each, and then choosing the shortest route found. This is an example of an “exhaustion” algorithm (because we exhaust all the possibilities before reaching our conclusion). Any implementation of this algorithm in code can be expected to be of factorial order.

Or Travelling Salesperson Problem or Travelling Saleswoman Problem.

At the end of Section 1 we suggested that you may be wondering if there was a better algorithm for finding the shortest route. The answer is — yes and no. There are algorithms that perform more quickly than an exhaustive search, but they do not necessarily provide optimal solutions to the problem. Such algorithms may be “nearly as good” for practical purposes, but they do not address exactly the same problem (that of finding the strictly shortest route).

But if we are prepared to relax our requirement that we find the shortest route, and settle for finding a “good” route in a reasonable time, then there is an algorithm that is guaranteed to give you a route whose length is no more than twice the minimum possible. Of course, one day, someone may improve on this!

The TSP is an example of an NP-Complete problem. We shall say a little more about such problems in Unit 16.

See, e.g., Christofides, N., “Worst-case analysis of a new heuristic for the travelling salesman problem”, in J. F. Traub (ed) *Algorithms and Complexity: New Directions and Recent Results*, 1976, Academic Press, p. 441.

Summary of Unit 15

1. The time required to run a program depends on many factors. The task of route planning provides a problem to which a computer may have difficulty in arriving at a solution in a feasible time.
2. Time complexity is measured in such a way as to make it a platform independent description of the cost of running software. It is related to the actual time required to run a program, but is not equivalent to it. Comparing the efficiency of different algorithms, each of which solves the same problem in a different way, allows us to choose the best algorithm for our application. Our choice may be guided by knowledge of the form of the input data. We need to distinguish best, average and worst cases for efficiency of algorithms — here, we have concentrated on worst cases.
3. For a given implementation (call it A), we defined its time complexity function, written as $T_A(n)$, as the number of statement executions when the implementation is run for an input of size n , in the worst case. Examination of the code in an implementation may enable us to suggest a formula for the time complexity.
4. Given a recursive algorithm, we can derive a recurrence system for its time complexity function. Given a recurrence system and an unproven idea of a likely time complexity function, $T(n)$, we can use mathematical induction to determine whether $T(n)$ is valid.

For a suitable recurrence system, we can derive a solution by forming a list of equations from the recurrence system in the appropriate way, and summing each side.

5. The behaviour of functions for large inputs can be compared informally by examining their graphs. We concentrated on certain representative functions. Exponential and factorial functions grow very quickly — if an algorithm has an exponential or factorial time complexity, it may not be possible to run it for a large inputs in any feasible amount of time. Polynomial functions grow faster as the degree of the polynomial increases. The logarithmic function grows more slowly than a linear function.

We then gave a more rigorous definition of what it means for a function to be growing at a certain rate. If a function $g(n)$ asymptotically dominates a function $f(n)$, we say that $f(n)$ is $O(g(n))$, and this means that $g(n)$ provides an upper bound on the way $f(n)$ grows when n is large. The order of a function is written as $\Theta(g(n))$ (for a suitable function $g(n)$). Typical orders of time complexity functions include $\log n$, n , $n \log n$ and n^2 . We can find the order of a time complexity function given by a formula by finding its dominant term and then removing any multiplying

factors. Care needs to be taken when interpreting the order of a time complexity function, as the constants involved are not usually known. We should bear in mind that we have made simplifying assumptions, and watch out for conditions that may make those assumptions unreasonable.

Solutions to the exercises

Solution 1.1

It is reasonable to assume that the time needed to drill 40 holes will be a fixed amount, and so the only quantity that can be optimised is the time required to move the robot arm between locations. Then this problem is directly comparable to the problem of the supermarket delivery driver. The robot arm has 40 locations (“houses”) to visit, stopping to drill a hole (“make a delivery”) at each. If we use the algorithm for supermarket delivery discussed in the text, we would find the distances between all pairs of hole locations, calculate the total length of each route that visits every hole location once, and pick the shortest route. However, as we saw for 40 home deliveries, with 40 hole locations there are so many possible routes that it is not feasible to calculate all their lengths, so this algorithm is not a feasible way to determine an optimum strategy.

Solution 2.1

(a) The number of divisions that will be performed by this algorithm will depend on the number z — and is likely to be larger if z is larger. So the magnitude of the input number, z , is an appropriate measure of how much work might have to be done.

(b) The best case will be when z is divisible by the first number that we try as a factor. That will be 2, and so the “best case” here is when z is even. In that case, only one division is required to arrive at the return value of **false**.

The worst case will occur when we need to try every number between 2 and $z - 1$ (inclusive). That will involve a total of $z - 2$ divisions, and will occur when z is prime. So this is the “worst case” here.

(As is typical, it is much harder to tell what the average amount of work will be, but it is true to say that the average amount of work is also related closely to the value z .)

Solution 2.2

For the unsorted list in (a) there is no relationship between an account number and where it will appear in the list of users who are logged on. Any search to determine if a user is logged on must consider each entry in the list, one at a time, until the status of the account is determined. The time required to search will be proportional to the number of users who are logged on.

In the second case, we need to locate the account holder in the sorted list forming the first column of the table. Searching in a sorted list is much quicker than searching a list item-by-item. We discussed this briefly in Unit 6, Section 4, and will look at it more carefully in Subsection 3.2 of this unit.

However, in the first case we only need to search through the users who are actually logged on, while the list in the second case involves all account holders. So which is preferable may also be related to the proportion of account holders who are typically logged on.

There is also a difference in the storage requirement of the two approaches — the first will require less, possibly a great deal less. If storage is an issue, then this could influence the choice of approach.

You may have thought of a third approach: to use a sorted vector containing the account holders who are logged on. In comparison with (a), this will require longer to add a new account holder when they log on, but less time to determine whether a particular account holder is logged on (or to find a particular account holder in the sequence when they log off, so that they can be removed from the sequence).

Solution 3.1

(a) Each of the statements (1), (2), (3) and (6) is executed exactly once, while statement (5) is executed n times and (4) is executed $n + 1$ times.

So $T_{itFact}(n) = 1 + 1 + 1 + (n + 1) + n + 1 = 2n + 5$.

(b) The values returned are unchanged — we have simply saved an unnecessary multiplication by 1. In this case, statements (4) and (5) are each executed one less time. So, in this case, $T_{itFact}(n) = 1 + 1 + 1 + (n) + (n - 1) + 1 = 2n + 3$. (If n is 1, the initialisation in (3) sets *result* to 1, and the loop body is never executed. The formula gives five statement executions in the case $n = 1$, which is correct.)

Solution 3.2

(a) A trace showing every statement execution when the input n is 3 is given below.

When n is 3.

Statement	Comment	<i>fac</i>
(1)		—
(2)	$3 == 1$ is false	—
(4)	pending evaluation	$3 \times \text{FACTORIAL}(2)$

The pending evaluation of $\text{FACTORIAL}(3)$ will be found using *recFact*. So we now need to trace evaluation of $\text{FACTORIAL}(n)$ using *recFact* when the input is 2.

When n is 2.

Statement	Comment	<i>fac</i>
(1)		—
(2)	$2 == 1$ is false	—
(4)	pending evaluation	$2 \times \text{FACTORIAL}(1)$

Now trace the evaluation of $\text{FACTORIAL}(1)$, which is the stopping case.

When n is 1.

Statement	Comment	<i>fac</i>
(1)		—
(2)	$1 == 1$ is true	—
(3)		1

Then 1 is returned as $\text{FACTORIAL}(1)$ — an execution of statement (5);
then $2 \times 1 = 2$ is returned as $\text{FACTORIAL}(2)$ — an execution of statement (5);
then $3 \times 2 = 6$ is returned as $\text{FACTORIAL}(3)$ — an execution of statement (5).

So there are a total of 12 statement executions for this input.

(b) (i) In the stopping case (when n is 1), statements (1), (2), (3) and (5) are executed — a total of 4.

(ii) For $n > 1$, there will be $n - 1$ pending recursive calls when we reach the stopping case of $n = 1$.

For $n > 1$, there are executions of statements (1), (2), (4) and, after the pending recursive call is completed, (5) — a total of 4.

(iii) So we can see that

$$T_{recFact}(n) = 4 + 4(n - 1) = 4n.$$

Solution 4.1

(a) When $n > 1$, recFact involves a recursive call. In this case, four statements are executed; (1), (2), (4) and (5), together with the statements required for the recursive call at (4). The stopping case is $n = 1$, when statements (1), (2), (3) and (5) are executed. This gives the recurrence system below.

$$\begin{aligned} T_{\text{recFact}}(1) &= 4 \\ T_{\text{recFact}}(n) &= T_{\text{recFact}}(n-1) + 4 \quad (n > 1) \end{aligned}$$

(b) We will give a quick check, rather than a formal inductive proof. First, check that the formula works for the stopping condition. With $n = 1$, the formula gives $4 \times 1 = 4$, which fits the stopping value for $T_{\text{recFact}}(1)$ from the recurrence.

Also, the inductive hypothesis would be that $T_{\text{recFact}}(k-1) = 4(k-1)$, and if we put this into the right-hand side of the recurrence, we get

$$T_{\text{recFact}}(k) = 4(k-1) + 4 = 4k - 4 + 4 = 4k.$$

So if the formula is correct for $k-1$ then it is also correct for k .

Solution 4.2

We use the procedure considered in Subsection 4.2. Write out a list of equations, using the recursive part of the recurrence repeatedly, and ending with the stopping condition.

$$\begin{aligned} T(n) &= T(n-1) + 2n \\ T(n-1) &= T(n-2) + 2(n-1) \\ T(n-2) &= T(n-3) + 2(n-2) \\ &\vdots \\ T(2) &= T(1) + 2 \times 2 \\ T(1) &= 1 \end{aligned}$$

Now we sum the left- and right-hand sides of each equation to form a new equation.

$$\begin{aligned} T(n) + T(n-1) + T(n-2) + \dots + T(2) + T(1) &= \\ T(n-1) + 2n + T(n-2) + 2(n-1) + T(n-3) + 2(n-2) + \dots + T(1) + 2 \times 2 + 1 & \end{aligned}$$

This allows us to subtract common terms from each side, to leave us with $T(n)$ on the left.

$$\begin{aligned} T(n) &= 2n + 2(n-1) + 2(n-2) + \dots + 2 \times 2 + 1 \\ &= 1 + 2(2 + 3 + \dots + (n-2) + (n-1) + n) \\ &= 1 - 2 + 2(1 + 2 + 3 + \dots + (n-2) + (n-1) + n) \\ &= 1 - 2 + 2(n(n+1)/2) \quad (\text{using the formula given in the question}) \\ &= n(n+1) - 1 \\ &= n^2 + n - 1 \end{aligned}$$

Solution 5.1

(a) Formally, this means that $f(n) \leq c \times n^2$ for some positive constant c , when n is large enough. Informally, it means that the behaviour of $f(n)$ for large n is (in a general way) no worse than n^2 .

(b) It is $\Theta(1)$. (This constant may be big, but it is still a constant.)

(c) Yes. But it is also $O(n^4)$ and $O(n^3)$ and $O(n^6)$. The most useful information is that which gives us the tightest bound, i.e. that $f(n)$ is $O(n^3)$. Indeed, since the formula for $f(n)$ includes a term in n^3 , we can be sure that $f(n)$ is $\Theta(n^3)$.

(d) The reason is that from this information alone we do not know what the multiplying factors c were in each case, nor from what values of N each function is bounded above by the quadratic or cubic polynomial, respectively. In fact, c could be large in one case and small in the other. Similarly, N could be large in one case and small in the other. In principle, it could be the case that one or both of c or N is so large that for all inputs of a realistic size, the $\Theta(n^2)$ function is larger than the $\Theta(n^3)$ function. Having said this, in common usage it is true to say that $\Theta(n^2)$ is better than $\Theta(n^3)$, because typically the sizes of the constants are not important.

Solution 5.2

In the formula for $f(n)$, the dominant term is $n \log n$. (This appears furthest to the right in Table 5.2.) So $f(n)$ is $\Theta(n \log n)$. In the formula for $g(n)$, the dominant term is n^2 . So $g(n)$ is $\Theta(n^2)$.

Since n^2 is “worse” than $n \log n$ (it appears further to the right in Table 5.2), we would expect algorithm A, with time complexity function $f(n)$, to be the more efficient for large inputs.

Index

- $O(f(n))$, 41
- $\Theta(f(n))$, 42
- algorithm, 9
- asymptotic behaviour (of a function), 35
- asymptotic domination, 40
- average case, 10
- best case, 10
- big Oh, 41
- binary search, 22
- computationally hard, 36
- linear search, 20
- NP-Complete problem, 47
- platform independent, 4
- polynomial, 37
 - constant, 37
 - cubic, 37
 - degree of , 37
 - linear, 37
 - quadratic, 37
- sequential search, 20
- time complexity function, 12
 - order of a, 42
- time efficiency function, 12
- worst case, 10

